HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

-----□□□□□-----

BÁO CÁO BÀI TẬP

Đề tài: Xây dựng hệ thống

quản lý lớp nhà thông minh Smart House

Giảng viên : **Nguyễn Quốc Uy**

Sinh viên: Phạm Bảo Lộc – B23DCCN493

Hà Nội - 2026

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG KHOA CÔNG NGHỆ THÔNG TIN 1 1
Hà Nội - 2026 1
1. Giới thiệu về dự án 4
1.1. Lý do lựa chọn đề tài 4
1.2. Mục tiêu dự án 4
1.3. Ý nghĩa dự án 4
1.4. Phạm vi dự án 4
2. Kiến trúc hệ thống tổng quát 5
2.1. Lớp thiết bị (Hardware Layer) 5
2.2. Lớp mạng và truyền thông (Network & MQTT Layer) 6
2.3. Lớp Backend và Database 6
2.4. Lớp giao diện người dùng (Frontend Layer) 8
3. Phân tích yêu cầu **10**
3.1. Actor 10
3.2. Usecase chính 11
3.3. Mô tả usecase 11
3.3.1. Xem dữ liệu cảm biến realtime 11
3.3.2. Điều khiển thiết bị 15
3.3.3. Xem dữ liệu cảm biến 20
3.3.4. Xem lịch sử hoạt động của thiết bị 23
3.3.5. Biểu đồ dữ liệu cảm biến 27
3.3.6. Xem thông tin cá nhân người dùng 30
4.1. Kết quả triển khai hệ thống 33
4.1.1. Triển khai phần cứng và chương trình ESP32 33
4.1.2. Triển khai MQTT Broker 34
4.1.3. Triển khai cơ sở dữ liệu 35
4.1.4. Triển khai Backend Spring Boot 36
4.1.5. Triển khai Frontend React 37
4.2. Kết quả giao diện và chức năng 37
4.2.1. Màn hình Dashboard 37
4.2.2. Màn hình DataSensor 38
4.2.3. Màn hình History 39
4.2.4. Màn hình Profile 39
4.3. Kiểm thử hệ thống 40
4.4. Đánh giá kết quả 48
5.1. Kết quả đạt được 49
5.2. Hạn chế của hệ thống 49
5.4. Kết luận 50

Chương 1: Tổng quan về dự án

1. Giới thiệu về dự án

Dự án "Hệ thống Nhà thông minh – Smart Home" được xây dựng nhằm giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển thiết bị thông qua giao diện Web. ESP32 thu thập dữ liệu cảm biến, trao đổi với Backend qua MQTT. Backend xử lý, lưu trữ dữ liệu trong MySQL và cập nhật thông tin lên giao diện theo thời gian thực.

Hệ thống cho phép người dùng theo dõi số đo, quan sát biểu đồ, nhận cảnh báo vượt ngưỡng, bật/tắt thiết bị và tra cứu lịch sử.

1.1. Lý do lựa chọn đề tài

Giám sát môi trường và điều khiển thiết bị từ xa là những ứng dụng phổ biến của IoT trong nhà thông minh. Đề tài được lựa chọn nhằm vận dụng kiến thức về vi điều khiển, cảm biến, giao thức truyền thông và lập trình Web để xây dựng một mô hình có khả năng hoạt động thực tế.

1.2. Mục tiêu dự án

- Thu thập nhiệt độ, độ ẩm và ánh sáng theo chu kỳ.
- Hiển thị số đo, biểu đồ và cảnh báo vượt ngưỡng trên Dashboard.
- Điều khiển thiết bị từ giao diện Web và cập nhật kết quả theo phản hồi của ESP32.
- Lưu trữ, tìm kiếm, lọc và phân trang dữ liệu cảm biến, lịch sử điều khiển.
- Thông báo mất kết nối và khôi phục trạng thái thiết bị khi ESP32 kết nối lại..

1.3. Ý nghĩa dự án

Dự án minh họa quá trình xây dựng một hệ thống IoT từ thu thập dữ liệu, truyền thông, xử lý và lưu trữ đến hiển thị và điều khiển. Qua đó, người thực hiện rèn luyện kỹ năng tích hợp phần cứng với phần mềm, xử lý phản hồi bất đồng bộ và kiểm thử hệ thống.

1.4. Phạm vi dự án

- **Hệ thống được triển khai ở quy mô mô hình thử nghiệm, sử dụng một ESP32 DevKit V1, cảm biến DHT11, module quang trở LDR và ba LED mô phỏng đèn, quạt, điều hòa. Giá trị ánh sáng được quy đổi thành lux ước tính, chưa được hiệu chuẩn.**
- **Phần mềm gồm chương trình ESP32, Mosquitto MQTT Broker, Backend Spring Boot, cơ sở dữ liệu MySQL và Frontend React. Giao diện có bốn trang: Dashboard, DataSensor, History và Profile.**

- **Dự án chưa triển khai điều khiển thiết bị điện gia dụng thực tế, xác thực người dùng hoặc quản lý nhiều ngôi nhà.**

Chương 2: Phân tích và thiết kế hệ thống

2. Kiến trúc hệ thống tổng quát

Hệ thống **Nhà Thông Minh – Smart Home** được thiết kế theo mô hình kiến trúc nhiều lớp, kết hợp giữa thiết bị IoT, mạng truyền thông, giao thức MQTT, Backend, cơ sở dữ liệu và giao diện Web. Kiến trúc này cho phép hệ thống thu thập các thông số môi trường trong nhà như **nhiệt độ, độ ẩm và cường độ ánh sáng**, truyền dữ liệu tới máy chủ để xử lý và lưu trữ, sau đó hiển thị cho người dùng trên giao diện Web.

Bên cạnh chức năng giám sát môi trường, hệ thống còn hỗ trợ người dùng theo dõi trạng thái và điều khiển các thiết bị trong nhà. Các thành phần trong hệ thống được tổ chức thành các lớp riêng biệt nhưng có sự liên kết với nhau thông qua mạng và các giao thức truyền thông.

Kiến trúc tổng quát của hệ thống bao gồm các thành phần chính sau:

2.1. Lớp thiết bị (Hardware Layer)

Đây là lớp trực tiếp tương tác với môi trường thực tế trong nhà và thực hiện việc thu thập dữ liệu từ các cảm biến.

Thành phần xử lý trung tâm của hệ thống là ESP32. Vi điều khiển có nhiệm vụ kết nối với mạng Wi-Fi, đọc dữ liệu từ các cảm biến, xử lý dữ liệu ban đầu và truyền các thông số thu được lên hệ thống thông qua giao thức MQTT.

Các thiết bị chính tại lớp phần cứng bao gồm:

- **Cảm biến DHT11:** Được sử dụng để đo nhiệt độ và độ ẩm trong không gian nhà ở.
- **Cảm biến ánh sáng LDR (quang trở):** Có nhiệm vụ ghi nhận mức độ ánh sáng của môi trường.
- **ESP32:** Đóng vai trò là bộ điều khiển trung tâm, thực hiện việc đọc dữ liệu cảm biến, kết nối mạng và trao đổi dữ liệu với MQTT Broker.
- **Các thiết bị điều khiển:** Được sử dụng để mô phỏng các thiết bị điện trong Nhà Thông Minh và có thể nhận lệnh bật/tắt từ hệ thống.

Trong quá trình hoạt động, ESP32 tiến hành đọc các giá trị nhiệt độ, độ ẩm và ánh sáng theo chu kỳ được thiết lập. Sau khi nhận được dữ liệu, các giá trị được tổng hợp thành bản tin và gửi lên MQTT Broker thông qua kết nối Wi-Fi.

Ở chiều ngược lại, ESP32 cũng có thể nhận các lệnh điều khiển thiết bị được gửi từ hệ thống thông qua MQTT và thực hiện thao tác tương ứng trên thiết bị phần cứng.

2.2. Lớp mạng và truyền thông (Network & MQTT Layer)

Lớp mạng và truyền thông đảm nhiệm việc trao đổi dữ liệu giữa ESP32, MQTT Broker và Backend.

ESP32 sử dụng kết nối Wi-Fi để tham gia vào mạng và giao tiếp với MQTT Broker. Dữ liệu cảm biến cũng như các lệnh điều khiển thiết bị được trao đổi thông qua giao thức MQTT (Message Queuing Telemetry Transport).

MQTT sử dụng mô hình Publish/Subscribe, phù hợp với các hệ thống IoT do có cấu trúc đơn giản, dung lượng bản tin nhỏ và khả năng trao đổi dữ liệu giữa nhiều thành phần mà không cần kết nối trực tiếp với nhau.

Trong hệ thống:

- **ESP32:** Đóng vai trò Publisher khi gửi dữ liệu nhiệt độ, độ ẩm và ánh sáng lên các MQTT Topic tương ứng. Đồng thời, ESP32 cũng có thể đóng vai trò Subscriber để nhận các lệnh điều khiển thiết bị.
- **MQTT Broker:** Là thành phần trung gian tiếp nhận các bản tin từ Publisher và chuyển tiếp dữ liệu tới những Subscriber đang đăng ký Topic tương ứng.
- **Spring Boot Backend:** Thực hiện Subscribe các Topic dữ liệu cảm biến để tiếp nhận thông tin từ ESP32. Khi người dùng gửi yêu cầu điều khiển thiết bị, Backend có thể Publish lệnh điều khiển lên MQTT Broker để ESP32 tiếp nhận và thực thi.

Nhờ sử dụng MQTT, dữ liệu giữa phần cứng và Backend có thể được trao đổi linh hoạt và phù hợp với yêu cầu cập nhật thường xuyên của hệ thống Nhà Thông Minh.

2.3. Lớp Backend và Database

Backend Spring Boot

Backend của hệ thống được xây dựng bằng **Java Spring Boot** và đóng vai trò trung tâm trong việc xử lý dữ liệu cũng như kết nối các thành phần của hệ thống.

Spring Boot là cầu nối giữa **MQTT Broker, Database và Frontend**, chịu trách nhiệm tiếp nhận dữ liệu từ thiết bị, xử lý các yêu cầu từ người dùng và cung cấp dữ liệu cần thiết cho giao diện Web.

Các nhiệm vụ chính của Backend bao gồm:

- Kết nối tới MQTT Broker.
- Subscribe các Topic chứa dữ liệu cảm biến.
- Tiếp nhận dữ liệu nhiệt độ, độ ẩm và ánh sáng được gửi từ ESP32.
- Phân tích và xử lý các bản tin dữ liệu nhận được.
- Kiểm tra tính hợp lệ của dữ liệu trước khi lưu.
- Lưu dữ liệu cảm biến vào cơ sở dữ liệu.
- Cung cấp REST API cho Frontend.
- Truy vấn dữ liệu cảm biến đã được lưu.
- Hỗ trợ tìm kiếm, lọc, sắp xếp và phân trang dữ liệu.
- Trả về dữ liệu cảm biến mới nhất cho Dashboard.
- Xử lý yêu cầu bật/tắt thiết bị từ người dùng.
- Publish lệnh điều khiển thiết bị thông qua MQTT Broker.
- Theo dõi và cập nhật trạng thái của thiết bị.
- Lưu lại lịch sử các thao tác điều khiển.
- Cập nhật dữ liệu mới tới Frontend khi hệ thống có cơ chế truyền dữ liệu thời gian thực.

Việc sử dụng Spring Boot giúp Backend được tổ chức thành các thành phần như **Controller, Service và Repository**, từ đó tách biệt rõ chức năng xử lý request, nghiệp vụ và truy cập dữ liệu. Điều này giúp hệ thống dễ phát triển, bảo trì và mở rộng thêm các chức năng sau này.

Database

Hệ thống sử dụng **MySQL** để lưu trữ các dữ liệu liên quan đến cảm biến, thiết bị và hoạt động của hệ thống Nhà Thông Minh.

Database có nhiệm vụ lưu trữ các thông tin chính như:

- Thông tin các loại cảm biến.
- Giá trị nhiệt độ.
- Giá trị độ ẩm.

- Giá trị ánh sáng.
- Thời gian ghi nhận dữ liệu.
- Thông tin các thiết bị trong hệ thống.
- Trạng thái hiện tại của thiết bị.
- Lịch sử bật/tắt thiết bị.
- Trạng thái thực hiện của các lệnh điều khiển.
- Thông tin người dùng và các thông tin liên quan nếu có.

Dữ liệu cảm biến được lưu lại theo từng thời điểm giúp người dùng không chỉ theo dõi giá trị hiện tại mà còn có thể xem lại lịch sử và quan sát sự biến đổi của nhiệt độ, độ ẩm và ánh sáng trong nhà theo thời gian.

Ngoài ra, việc lưu lại lịch sử điều khiển thiết bị giúp người dùng kiểm tra được thiết bị nào đã được bật hoặc tắt, thời gian thực hiện và trạng thái xử lý của thao tác.

2.4. Lớp giao diện người dùng (Frontend Layer)

Lớp giao diện người dùng là nơi người dùng trực tiếp theo dõi và tương tác với hệ thống Nhà Thông Minh.

Frontend được xây dựng bằng **ReactJS**, có nhiệm vụ gửi các yêu cầu tới REST API do Spring Boot cung cấp, tiếp nhận dữ liệu trả về và hiển thị trên giao diện Web.

Thông qua Frontend, người dùng có thể theo dõi các thông số môi trường, quan sát biểu đồ dữ liệu, tra cứu lịch sử, xem trạng thái cũng như điều khiển các thiết bị trong nhà.

Hệ thống bao gồm các giao diện chính sau:

Dashboard

Dashboard là màn hình tổng quan của hệ thống Nhà Thông Minh.

Giao diện này hiển thị các thông số môi trường mới nhất gồm:

- **Nhiệt độ.**
- **Độ ẩm.**
- **Cường độ ánh sáng.**

Ngoài các giá trị hiện tại, Dashboard còn có thể hiển thị biểu đồ thể hiện sự thay đổi của các thông số cảm biến theo thời gian.

Dashboard đồng thời cung cấp khu vực hiển thị trạng thái và điều khiển các thiết bị trong nhà. Người dùng có thể thực hiện các thao tác bật hoặc tắt thiết bị trực tiếp trên giao diện.

DataSensor

Trang DataSensor được sử dụng để hiển thị danh sách các dữ liệu cảm biến đã được hệ thống thu thập và lưu trữ trong Database.

Người dùng có thể thực hiện các thao tác:

- Xem danh sách dữ liệu cảm biến.
- Lọc dữ liệu theo loại cảm biến.
- Tìm kiếm dữ liệu.
- Sắp xếp dữ liệu.
- Chọn số lượng bản ghi hiển thị.
- Phân trang dữ liệu.

Thông qua trang này, người dùng có thể tra cứu chi tiết các giá trị nhiệt độ, độ ẩm và ánh sáng đã được ghi nhận tại những thời điểm khác nhau.

History

Trang History dùng để hiển thị lịch sử hoạt động và điều khiển các thiết bị trong hệ thống.

Người dùng có thể:

- Xem thời gian thực hiện thao tác.
- Xem thiết bị được điều khiển.
- Xem hành động bật hoặc tắt thiết bị.
- Theo dõi trạng thái thực hiện của lệnh.
- Lọc lịch sử theo từng thiết bị.
- Tìm kiếm và sắp xếp dữ liệu.
- Phân trang danh sách lịch sử.

Chức năng này giúp người dùng dễ dàng kiểm tra các hoạt động đã xảy ra đối với các thiết bị trong Nhà Thông Minh.

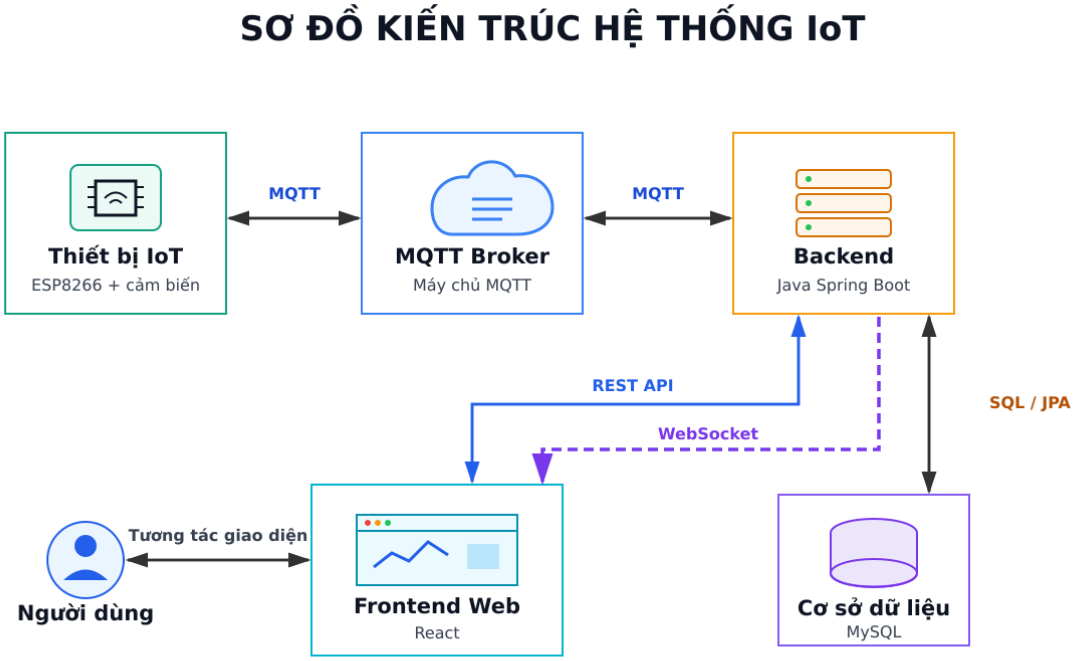
Profile

Trang Profile được sử dụng để hiển thị các thông tin liên quan đến người dùng và dự án.

Giao diện có thể bao gồm:

- Thông tin cá nhân của người dùng.
- Thông tin sinh viên.
- Thông tin API.
- Tài liệu của hệ thống.
- Liên kết mã nguồn dự án.
- Liên kết thiết kế giao diện.

Thông qua trang Profile, người dùng có thể xem các thông tin cá nhân cũng như truy cập nhanh tới các tài nguyên liên quan đến dự án.



Hình 1. Kiến trúc hệ thống

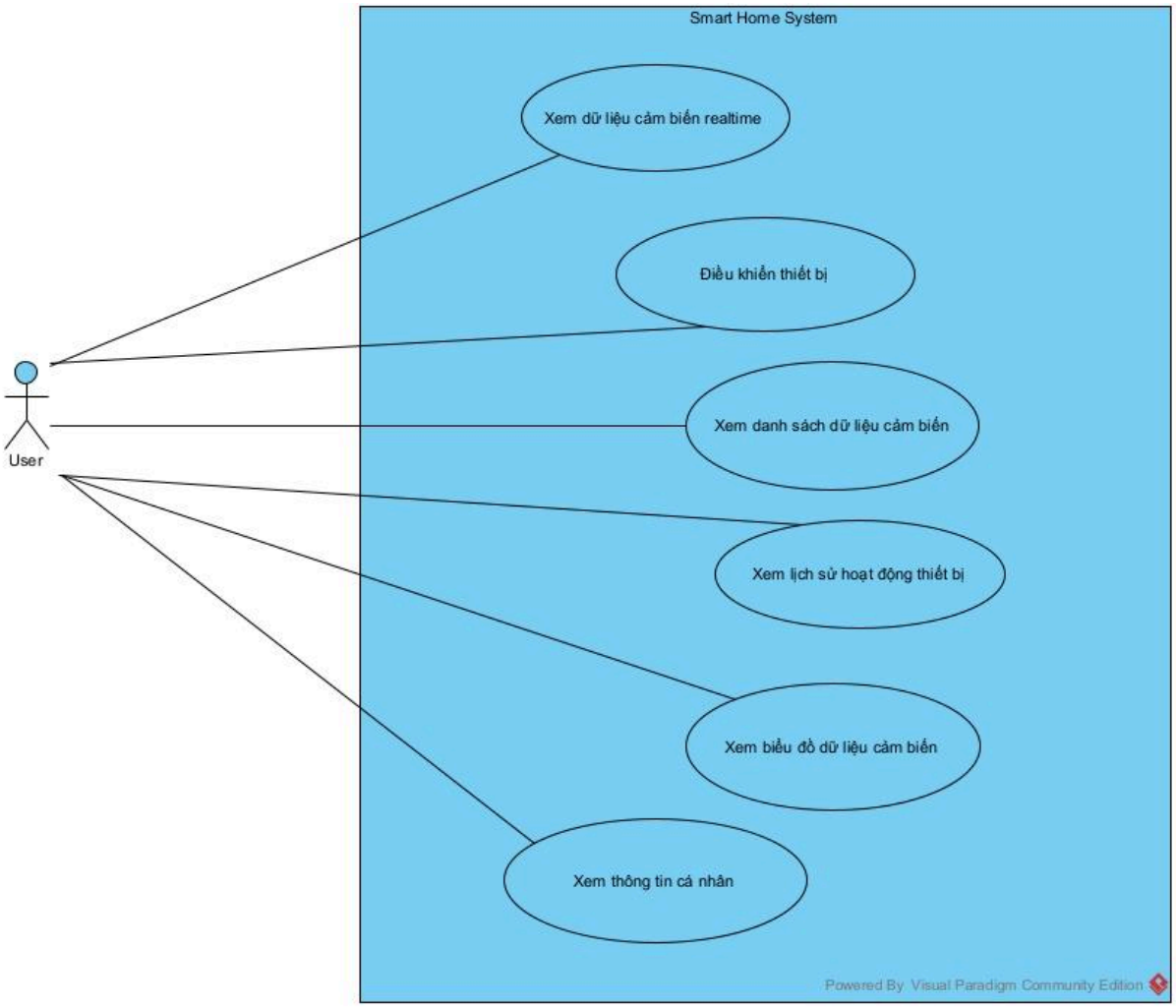
Chương 3: Thiết kế chi tiết

3. Phân tích yêu cầu

3.1. Actor

- Người dùng (user) là người sử dụng hệ thống thông qua giao diện Web.
- Vai trò:
  - Xem dữ liệu cảm biến theo thời gian thực
  - Xem lịch sử dữ liệu
  - Xem trạng thái và lịch sử thiết bị
  - Điều khiển bật/tắt thiết bị

3.2. Usecase chính



Hình 2. Use Case tổng quan

3.3. Mô tả usecase

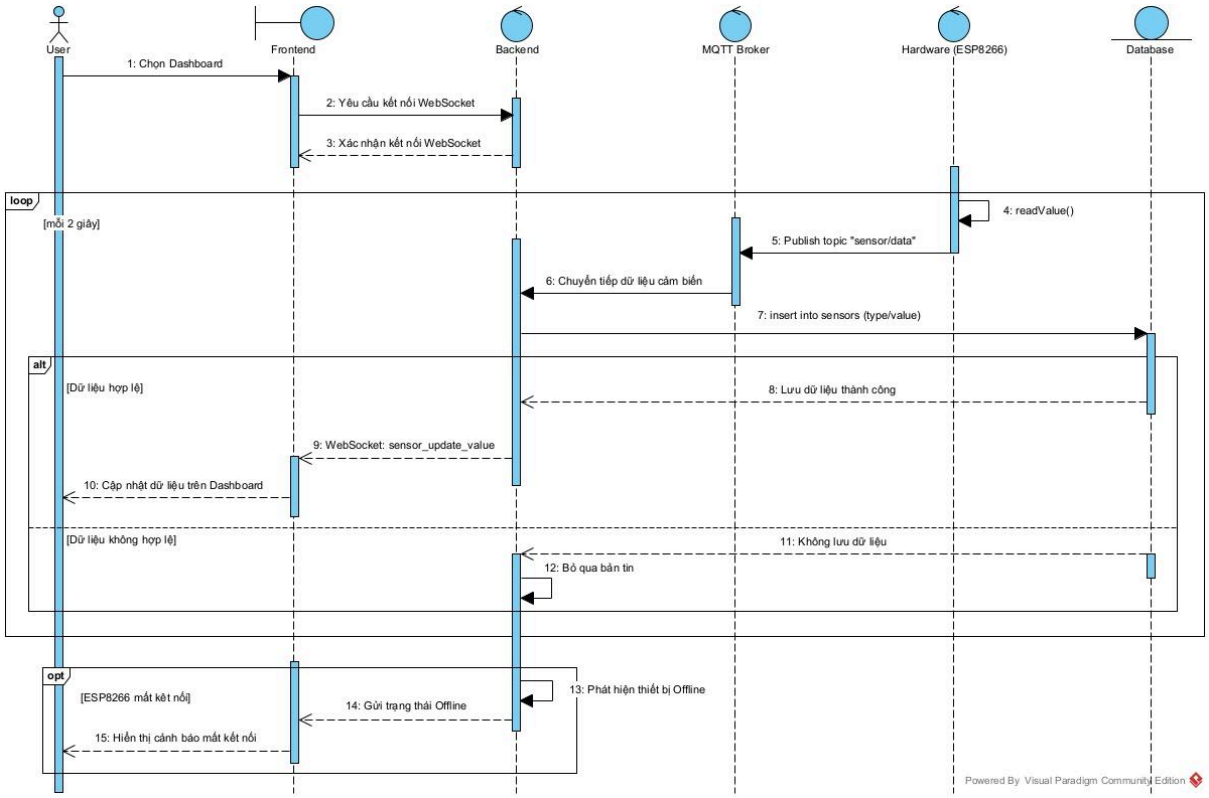
3.3.1. Xem dữ liệu cảm biến realtime

Tên chức năng	Xem dữ liệu cảm biến realtime
Mô tả	Chức năng cho phép người dùng theo dõi các thông số môi trường trong nhà gồm nhiệt độ, độ ẩm và cường độ ánh sáng
Đường dẫn	Người dùng đăng nhập vào hệ thống → chọn **Dashboard** trên thanh điều hướng.
Tác nhân	Người dùng đã đăng nhập vào hệ thống.
Luồng cơ bản	**Bước 1:** Người dùng đăng nhập hệ thống và truy cập vào màn hình Dashboard **Bước 2:** Frontend khởi tạo giao diện Dashboard, gửi yêu cầu tới Backend để lấy dữ liệu cảm biến gần nhất và thiết lập kết nối WebSocket để nhận dữ liệu mới. **Bước 3:** ESP32 đọc dữ liệu từ cảm biến DHT11 và cảm biến ánh sáng LDR theo chu kỳ định trước. **Bước 4:** ESP32 đóng gói các giá trị nhiệt độ, độ ẩm và ánh sáng, sau đó Publish dữ liệu lên MQTT Broker thông qua mạng Wi-Fi. **Bước 5:** Backend đã Subscribe Topic dữ liệu cảm biến nên nhận được bản tin do MQTT Broker chuyển tiếp. **Bước 6:** Backend kiểm tra, xử lý dữ liệu và lưu bản ghi cảm biến vào Database. **Bước 7:** Backend gửi dữ liệu cảm biến mới nhất tới Frontend thông qua kết nối WebSocket. **Bước 8:** Frontend nhận dữ liệu và cập nhật các giá trị nhiệt độ, độ ẩm và ánh sáng trên Dashboard mà không cần người dùng tải lại trang.
Luồng ngoại lệ	**Trường hợp 1 – ESP32 mất kết nối Wi-Fi:**

	1. ESP32 không thể kết nối tới mạng Wi-Fi. 2. Dữ liệu cảm biến không được Publish lên MQTT Broker. 3. Backend không nhận được dữ liệu mới từ thiết bị. 4. Dashboard giữ lại giá trị gần nhất và có thể hiển thị trạng thái thiết bị Offline. **Trường hợp 2 – MQTT Broker mất kết nối:** 1. ESP32 không thể Publish dữ liệu hoặc Backend không thể nhận bản tin từ Broker. 2. Luồng cập nhật dữ liệu thời gian thực bị gián đoạn. 3. Hệ thống giữ lại dữ liệu gần nhất và hiển thị trạng thái mất kết nối nếu cần. **Trường hợp 3 – Dữ liệu cảm biến không hợp lệ:** 1. Backend nhận bản tin dữ liệu từ MQTT Broker. 2. Backend thực hiện kiểm tra các giá trị cảm biến. 3. Nếu dữ liệu không hợp lệ, bản tin bị bỏ qua và không được lưu vào Database. 4. Hệ thống tiếp tục chờ và xử lý bản tin tiếp theo. **Trường hợp 4 – Mất kết nối WebSocket:** 1. Backend vẫn có thể tiếp tục nhận và lưu dữ liệu cảm biến từ MQTT Broker. 2. Frontend không nhận được dữ liệu mới trong thời gian kết nối bị gián đoạn. 3. Khi kết nối được khôi phục, Frontend tiếp tục nhận dữ liệu mới hoặc gửi yêu cầu lấy lại giá trị gần nhất.
Điều kiện trước	Người dùng đã đăng nhập vào hệ thống. ESP32 và các cảm biến đang hoạt động. ESP32 đã kết nối Wi-Fi và MQTT Broker.

	Backend, Database và dịch vụ WebSocket đang hoạt động.
Điều kiện sau	Dữ liệu nhiệt độ, độ ẩm và ánh sáng mới nhất được lưu vào Database và hiển thị trên Dashboard. Khi có bản tin cảm biến mới, giao diện được cập nhật tự động theo thời gian thực.

Sequence diagram:



Hình 3. Sequence diagram chức năng Xem dữ liệu cảm biến realtimes

Mô tả luồng chức năng:

Giai đoạn 1: Thiết lập kết nối thời gian thực

- **Bước 1:** Người dùng truy cập vào màn hình **Dashboard** trên giao diện Web.
- **Bước 2:** Frontend gửi yêu cầu thiết lập kết nối WebSocket tới Backend thông qua WebSocket endpoint **/ws/sensor**.
- **Bước 3:** Backend chấp nhận yêu cầu và xác nhận kết nối WebSocket đã được thiết lập thành công. Sau khi kết nối hoàn tất, Frontend duy trì kết nối để nhận dữ liệu cảm biến mới từ Backend theo thời gian thực.

Giai đoạn 2: Thu thập và truyền dữ liệu cảm biến

Quá trình thu thập dữ liệu được thực hiện lặp lại theo chu kỳ **2 giây**.

- **Bước 4:** ESP32 đọc dữ liệu từ các cảm biến để lấy giá trị **nhiệt độ, độ ẩm và cường độ ánh sáng** trong nhà.
- **Bước 5:** ESP32 đóng gói dữ liệu cảm biến và Publish bản tin lên MQTT Broker thông qua Topic **sensor/data**.
- **Bước 6:** Backend đã Subscribe Topic **sensor/data**, vì vậy MQTT Broker chuyển tiếp bản tin cảm biến từ ESP32 tới Backend.
- **Bước 7:** Backend tiếp nhận và kiểm tra tính hợp lệ của dữ liệu cảm biến.

Giai đoạn 3: Xử lý và cập nhật dữ liệu

- **Trường hợp dữ liệu hợp lệ:**
    - **Bước 8:** Backend lưu dữ liệu nhiệt độ, độ ẩm và ánh sáng vào bảng **DataSensors** trong cơ sở dữ liệu.
    - **Bước 9:** Database xác nhận quá trình lưu dữ liệu thành công.
    - **Bước 10:** Backend gửi dữ liệu cảm biến mới tới Frontend thông qua WebSocket endpoint **/ws/sensor** với loại sự kiện **sensor_update**.
    - **Bước 11:** Frontend tiếp nhận dữ liệu và cập nhật các giá trị nhiệt độ, độ ẩm và ánh sáng trên Dashboard.
- **Trường hợp dữ liệu không hợp lệ:**
    - **Bước 12:** Backend bỏ qua bản tin và không lưu dữ liệu vào Database.
    - **Bước 13:** Hệ thống tiếp tục chờ bản tin mới từ Topic **sensor/data** ở chu kỳ tiếp theo.

Giai đoạn 4: Xử lý khi thiết bị mất kết nối

- **Bước 14:** Nếu Backend không nhận được dữ liệu từ ESP32 trong khoảng thời gian quy định, hệ thống xác định thiết bị đang ở trạng thái **Offline**.
- **Bước 15:** Backend gửi trạng thái thiết bị tới Frontend thông qua WebSocket endpoint **/ws/sensor** với loại sự kiện **hardware_status**.
- **Bước 16:** Frontend nhận trạng thái và hiển thị cảnh báo mất kết nối trên Dashboard.

3.3.2. Điều khiển thiết bị

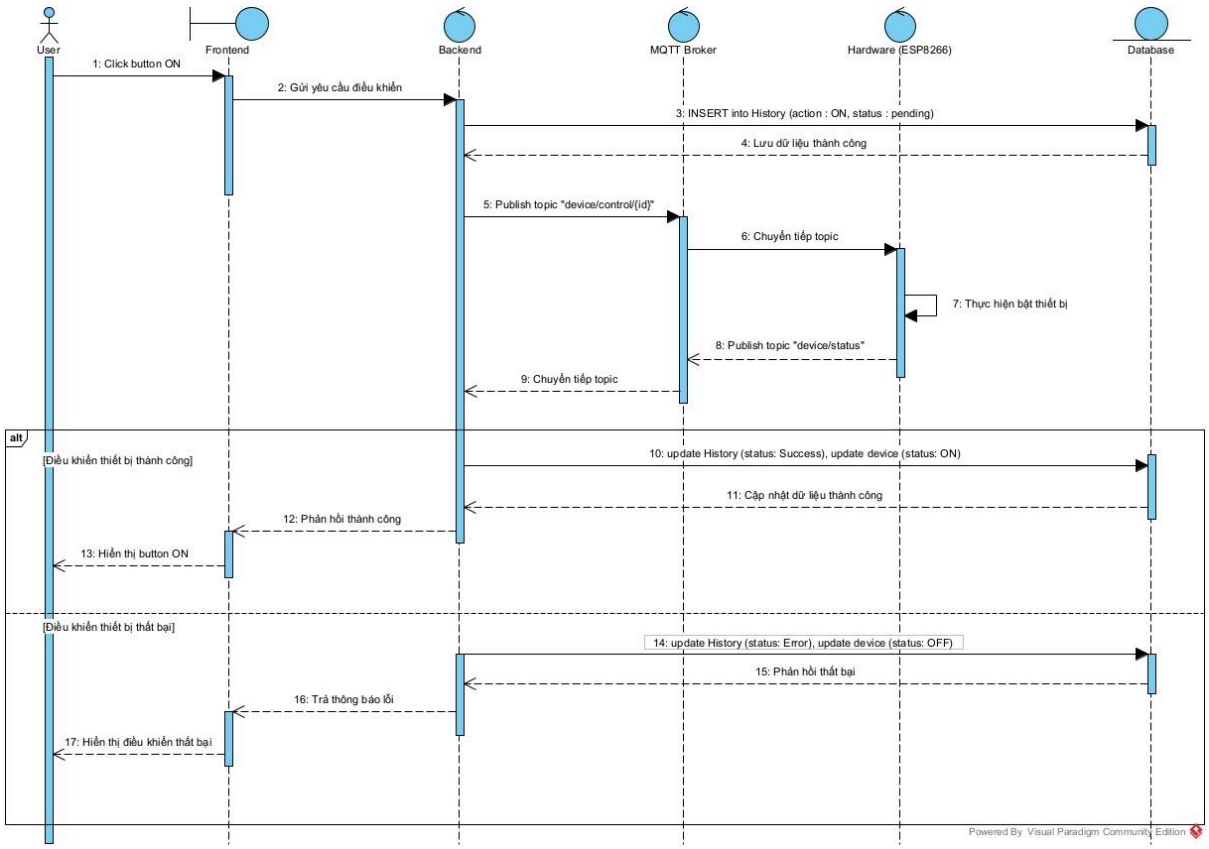
Tên chức năng	Xem dữ liệu cảm biến realtime

Mô tả	Cho phép người dùng thực hiện thao tác bật hoặc tắt các thiết bị trong hệ thống Nhà Thông Minh thông qua giao diện Dashboard..
Đường dẫn	Web → Dashboard → Khu vực điều khiển thiết bị
Tác nhân	Người dùng đã đăng nhập vào hệ thống.
Luồng cơ bản	**Bước 1:** Người dùng truy cập màn hình Dashboard và lựa chọn thiết bị cần điều khiển. **Bước 2:** Người dùng nhấn công tắc **ON/OFF** tương ứng với trạng thái mong muốn. **Bước 3:** Frontend gửi yêu cầu điều khiển thiết bị tới Backend, kèm theo mã thiết bị và trạng thái ON/OFF được lựa chọn. **Bước 4:** Backend tiếp nhận yêu cầu, kiểm tra thông tin thiết bị và tạo một bản ghi lịch sử điều khiển trong bảng **History** với trạng thái ban đầu là **Pending**. **Bước 5:** Backend Publish lệnh điều khiển lên MQTT Broker thông qua Topic điều khiển thiết bị. **Bước 6:** ESP32 đã Subscribe Topic điều khiển tương ứng nên nhận được lệnh từ MQTT Broker và thực hiện bật hoặc tắt thiết bị. **Bước 7:** Sau khi thực hiện lệnh, ESP32 gửi trạng thái mới của thiết bị lên MQTT Broker. **Bước 8:** MQTT Broker chuyển tiếp trạng thái thiết bị về Backend. **Bước 9:** Backend cập nhật trạng thái thiết bị trong bảng **Devices** thành ON hoặc OFF, đồng thời cập nhật trạng thái

	bản ghi trong bảng **History** từ **Pending** sang **Success**. **Bước 10:** Backend gửi kết quả điều khiển thành công về Frontend. **Bước 11:** Frontend cập nhật trạng thái công tắc trên Dashboard và hiển thị trạng thái mới của thiết bị cho người dùng.
Luồng ngoại lệ	**Trường hợp 1 – ESP32 không phản hồi:** Sau khi Backend gửi lệnh điều khiển qua MQTT Broker, nếu không nhận được trạng thái phản hồi từ ESP32 trong khoảng thời gian quy định, Backend xác định yêu cầu bị Timeout. Bản ghi trong bảng History được cập nhật thành **Error**, trạng thái thiết bị giữ nguyên và Frontend hiển thị thông báo điều khiển thất bại. **Trường hợp 2 – Mất kết nối MQTT Broker:** Nếu Backend không thể gửi lệnh điều khiển tới MQTT Broker, yêu cầu được xác định là thất bại. Backend cập nhật trạng thái bản ghi trong History thành **Error** và gửi thông báo lỗi về Frontend. **Trường hợp 3 – Yêu cầu điều khiển không hợp lệ:** Nếu thiết bị không tồn tại hoặc trạng thái điều khiển không hợp lệ, Backend từ chối yêu cầu, không gửi lệnh tới MQTT Broker và trả thông báo lỗi về Frontend.
Điều kiện trước	Người dùng đã đăng nhập vào hệ thống. Frontend và Backend đang hoạt động. ESP32 đã kết nối Wi-Fi và MQTT Broker. Thiết bị cần điều khiển đã được khai báo trong hệ thống. MQTT Broker và Database đang hoạt động.

Điều kiện sau	**Trường hợp thành công:** Thiết bị được chuyển sang trạng thái ON/OFF theo yêu cầu của người dùng; trạng thái trong bảng **Devices** được cập nhật; bản ghi trong **History** chuyển sang **Success**; Frontend hiển thị trạng thái mới của thiết bị. **Trường hợp thất bại:** Thiết bị giữ nguyên trạng thái trước đó, bản ghi History được cập nhật thành **Error** và Frontend hiển thị thông báo lỗi cho người dùng.

Sequence diagram:



Hình 5. Sequence diagram chức năng điều khiển thiết bị

Mô tả luồng chức năng:

Giai đoạn 1: Gửi và ghi nhận lệnh điều khiển

- **Bước 1:** Người dùng thao tác trên Dashboard và chọn trạng thái **ON** cho thiết bị cần điều khiển.
- **Bước 2:** Frontend gửi một HTTP Request **POST** tới Backend thông qua endpoint **/api/devices/control/{id}**
- **Bước 3:** Backend tiếp nhận yêu cầu, kiểm tra thông tin thiết bị và tạo một bản ghi lịch sử điều khiển trong bảng **History** với trạng thái ban đầu là **Pending**.
- **Bước 4:** Database lưu bản ghi lịch sử và phản hồi kết quả lưu thành công về Backend.
- **Bước 5:** Backend Publish lệnh điều khiển lên MQTT Broker thông qua Topic **device/control/{id}**.
- **Bước 6:** MQTT Broker tiếp nhận lệnh và chuyển tiếp bản tin tới ESP32 đang Subscribe Topic điều khiển tương ứng.
- **Bước 7:** ESP32 nhận lệnh và thực hiện thao tác bật hoặc tắt thiết bị theo yêu cầu.

Giai đoạn 2: Nhận trạng thái phản hồi từ thiết bị

- **Bước 8:** Sau khi thực hiện thao tác, ESP32 Publish trạng thái mới của thiết bị lên MQTT Broker thông qua Topic **device/status**.
- **Bước 9:** MQTT Broker chuyển tiếp trạng thái thiết bị tới Backend để xác nhận kết quả điều khiển.

Giai đoạn 3: Xử lý kết quả điều khiển

- **Trường hợp 1 – Điều khiển thành công:**
    - **Bước 10:** Khi Backend nhận được trạng thái phản hồi thành công từ ESP32, Backend cập nhật trạng thái thiết bị trong bảng **Devices** thành **ON/OFF** tương ứng và cập nhật bản ghi trong bảng **History** từ **Pending** sang **Success**.
    - **Bước 11:** Database thực hiện cập nhật và phản hồi kết quả thành công về Backend.
    - **Bước 12:** Backend trả phản hồi **200 OK** về Frontend, kèm theo trạng thái mới của thiết bị.
    - **Bước 13:** Frontend nhận kết quả và cập nhật trạng thái công tắc trên Dashboard để người dùng biết thao tác điều khiển đã được thực hiện thành công.
- **Trường hợp 2 – Không nhận được phản hồi hoặc xảy ra Timeout:**
    - **Bước 14:** Nếu sau khoảng thời gian quy định Backend không nhận được trạng thái phản hồi từ ESP32, yêu cầu điều khiển được xác định là thất bại.

- **Bước 15:** Backend cập nhật bản ghi trong bảng **History** từ **Pending** sang **Error**. Trạng thái của thiết bị trong bảng **Devices** được giữ nguyên theo trạng thái trước khi thực hiện yêu cầu.
- **Bước 16:** Backend trả phản hồi lỗi **504 Gateway Timeout** về Frontend.
- **Bước 17:** Frontend tiếp nhận thông báo lỗi, giữ nguyên trạng thái công tắc trước đó và hiển thị cảnh báo điều khiển thiết bị không thành công cho người dùng.

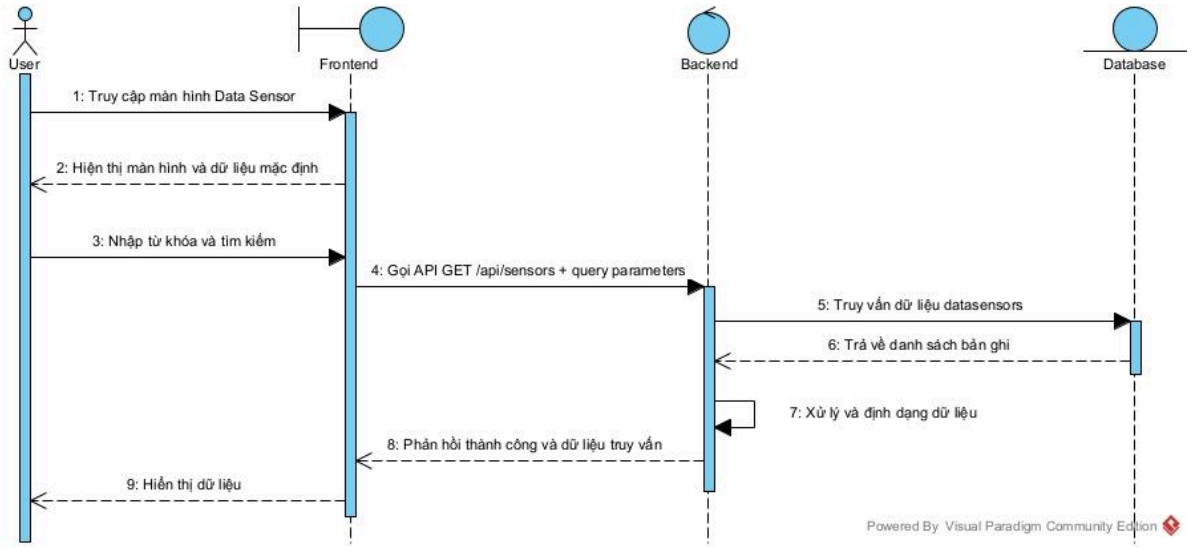
3.3.3. Xem dữ liệu cảm biến

Tên chức năng	Xem dữ liệu cảm biến
Mô tả	Cho phép người dùng xem danh sách các dữ liệu cảm biến đã được hệ thống thu thập và lưu trữ, bao gồm dữ liệu nhiệt độ, độ ẩm và ánh sáng trong nhà. Người dùng có thể thực hiện tìm kiếm, lọc theo loại cảm biến, sắp xếp, lựa chọn số lượng bản ghi hiển thị và phân trang dữ liệu trên giao diện **Data Sensor**
Đường dẫn	Web → Data Sensor
Tác nhân	Người dùng đã đăng nhập vào hệ thống.
Luồng cơ bản	**Bước 1:** Người dùng truy cập màn hình **Data Sensor** từ thanh điều hướng. **Bước 2:** Frontend hiển thị danh sách dữ liệu cảm biến hiện có. Người dùng có thể nhập từ khóa tìm kiếm, chọn loại cảm biến, lựa chọn thứ tự sắp xếp, số lượng bản ghi trên một trang hoặc chuyển sang trang dữ liệu khác. **Bước 3:** Frontend tổng hợp các điều kiện được lựa chọn thành các Query Parameter và gửi HTTP Request **GET /api/sensors** tới Backend.

	Bước 4: Backend tiếp nhận Request, kiểm tra các tham số tìm kiếm, lọc, sắp xếp và phân trang, sau đó thực hiện truy vấn dữ liệu cảm biến trong Database. **Bước 5:** Database truy vấn bảng **DataSensors** và các thông tin liên quan trong bảng **Sensors**, sau đó trả về danh sách bản ghi phù hợp cho Backend. **Bước 6:** Backend tiếp nhận kết quả, xử lý và định dạng dữ liệu trước khi trả về Frontend. **Bước 7:** Backend trả Response với mã trạng thái **200 OK**, bao gồm danh sách dữ liệu cảm biến và thông tin phân trang. **Bước 8:** Frontend nhận kết quả và hiển thị dữ liệu thành các hàng, cột trong bảng trên màn hình Data Sensor. **Bước 9:** Người dùng xem được danh sách dữ liệu nhiệt độ, độ ẩm và ánh sáng theo các điều kiện đã lựa chọn.
Luồng ngoại lệ	**Trường hợp 1 – Không tìm thấy dữ liệu:** Không có bản ghi nào phù hợp với các điều kiện tìm kiếm hoặc lọc. Backend trả về danh sách rỗng và Frontend hiển thị thông báo **"Không tìm thấy dữ liệu"**. **Trường hợp 2 – Tham số không hợp lệ:** Nếu các tham số như số trang, số lượng bản ghi hoặc điều kiện sắp xếp không hợp lệ, Backend từ chối xử lý yêu cầu và trả thông báo lỗi để Frontend hiển thị cho người dùng. **Trường hợp 3 – Lỗi truy vấn Database:** Nếu xảy ra lỗi khi truy xuất dữ liệu, Backend xử lý ngoại lệ và trả thông báo lỗi về Frontend. Frontend hiển thị cảnh báo không thể tải dữ liệu tại thời điểm hiện tại. .
Điều kiện trước	Người dùng đã đăng nhập và có quyền truy cập hệ thống. Frontend và Backend đang hoạt động.

	Database đang hoạt động. Hệ thống đã có dữ liệu cảm biến được thu thập và lưu trong Database.
Điều kiện sau	**Trường hợp thành công:** Danh sách dữ liệu cảm biến phù hợp được hiển thị trên màn hình Data Sensor, đồng thời các điều kiện tìm kiếm, lọc, sắp xếp và phân trang được áp dụng vào kết quả. **Trường hợp không có dữ liệu:** Frontend hiển thị thông báo không tìm thấy dữ liệu. **Trường hợp xảy ra lỗi:** Hệ thống hiển thị thông báo lỗi tương ứng và không hiển thị dữ liệu không hợp lệ.

Sequence diagram:



Hình 7. Sequence diagram chức năng xem dữ liệu cảm biến

Mô tả luồng chức năng:

Giai đoạn 1: Gửi yêu cầu tìm kiếm

- **Bước 1:** Người dùng truy cập màn hình **Data Sensor**, nhập từ khóa tìm kiếm hoặc lựa chọn các điều kiện như trường tìm kiếm, số lượng bản ghi hiển thị và tiêu chí sắp xếp, sau đó nhấn nút **Search**.
- **Bước 2:** Frontend tổng hợp các điều kiện người dùng đã chọn thành các **query parameter** và gửi HTTP Request **GET /api/sensors** tới Backend.

GET /api/sensors?searchField=id&search=12&sort=time&order=DESC&page=1&limit=4

Giai đoạn 2: Truy xuất và xử lý dữ liệu

- **Bước 3:** Backend tiếp nhận Request, đọc và kiểm tra các query parameter được gửi từ Frontend. Dựa trên các điều kiện này, Backend thực hiện truy vấn dữ liệu trong bảng **DataSensors** và liên kết với bảng **Sensors** để lấy thông tin loại cảm biến tương ứng.
- **Bước 4:** Database thực hiện truy vấn với các điều kiện tìm kiếm, sắp xếp và phân trang, sau đó trả về danh sách các bản ghi phù hợp cho Backend.
- **Bước 5:** Backend tiếp nhận kết quả từ Database và xử lý dữ liệu về định dạng phù hợp để trả về cho Frontend.

Giai đoạn 3: Trả kết quả và hiển thị

- **Bước 6:** Backend trả Response **200 OK** về Frontend, bao gồm danh sách dữ liệu cảm biến và thông tin phân trang.
- **Bước 7:** Frontend nhận dữ liệu và hiển thị các bản ghi trên bảng Data Sensor với các thông tin như **ID, thời gian ghi nhận, loại cảm biến và giá trị đo**.
- **Bước 8:** Người dùng có thể tiếp tục thay đổi điều kiện tìm kiếm, sắp xếp hoặc chuyển trang để thực hiện truy vấn mới.

3.3.4. Xem lịch sử hoạt động của thiết bị

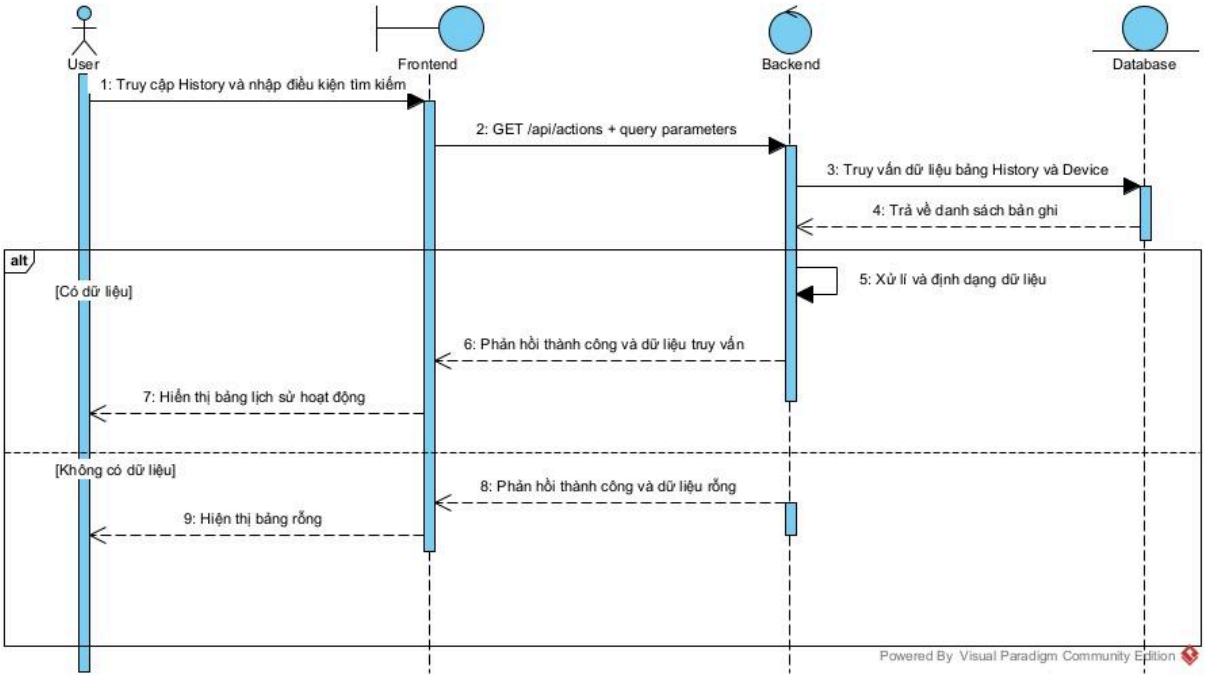
Tên chức năng	Xem lịch sử hoạt động của thiết bị

Mô tả	Chức năng cho phép người dùng xem lại các thao tác điều khiển đã được thực hiện đối với các thiết bị trong hệ thống Nhà Thông Minh. Thông tin hiển thị bao gồm thiết bị, hành động ON/OFF, trạng thái thực hiện và thời gian thao tác.
Đường dẫn	Web → History
Tác nhân	Người dùng đã đăng nhập vào hệ thống.
Luồng cơ bản	**Bước 1:** Người dùng truy cập màn hình **History** từ thanh điều hướng. **Bước 2:** Người dùng có thể nhập điều kiện tìm kiếm, lựa chọn thiết bị, tiêu chí sắp xếp, số lượng bản ghi hiển thị hoặc chuyển trang dữ liệu. **Bước 3:** Frontend tổng hợp các điều kiện thành query parameter và gửi yêu cầu lấy danh sách lịch sử hoạt động tới Backend. **Bước 4:** Backend tiếp nhận yêu cầu, kiểm tra các tham số và thực hiện truy vấn dữ liệu lịch sử trong Database. **Bước 5:** Database truy vấn bảng **History** và liên kết với bảng **Devices** để lấy thông tin thiết bị tương ứng, sau đó trả kết quả về Backend. **Bước 6:** Backend xử lý và định dạng danh sách lịch sử trước khi trả về Frontend. **Bước 7:** Backend trả danh sách lịch sử hoạt động cùng thông tin phân trang về Frontend. **Bước 8:** Frontend hiển thị các bản ghi trên bảng History để người dùng theo dõi.

Luồng ngoại lệ	**Trường hợp 1 – Không tìm thấy dữ liệu:** ● Database không có bản ghi phù hợp với điều kiện tìm kiếm hoặc lọc. ● Backend trả về danh sách rỗng. ● Frontend hiển thị thông báo **"Không tìm thấy dữ liệu"**. **Trường hợp 2 – Tham số tìm kiếm không hợp lệ:** ● Backend phát hiện tham số tìm kiếm, sắp xếp hoặc phân trang không hợp lệ. ● Backend từ chối xử lý yêu cầu và trả thông báo lỗi về Frontend. ● Frontend hiển thị thông báo để người dùng điều chỉnh lại điều kiện tìm kiếm. **Trường hợp 3 – Lỗi truy vấn Database:** ● Xảy ra lỗi trong quá trình truy xuất dữ liệu lịch sử. ● Backend xử lý ngoại lệ và trả thông báo lỗi về Frontend. ● Frontend hiển thị cảnh báo không thể tải dữ liệu tại thời điểm hiện tại.
Điều kiện trước	Người dùng đã đăng nhập và có quyền truy cập hệ thống. Frontend và Backend đang hoạt động. Database đang hoạt động. Hệ thống đã có dữ liệu lịch sử điều khiển được lưu trong bảng **History**.
Điều kiện sau	**Trường hợp thành công:** Danh sách lịch sử hoạt động phù hợp với điều kiện tìm kiếm được hiển thị trên màn hình History. Người dùng có thể tiếp tục tìm kiếm, lọc, sắp xếp hoặc chuyển trang.

	Trường hợp không có dữ liệu hoặc xảy ra lỗi: Frontend hiển thị thông báo tương ứng và không hiển thị dữ liệu không hợp lệ.

Sequence diagram:



Hình 9. Sequence diagram chức năng xem lịch sử hoạt động của thiết bị

Mô tả luồng chức năng:

Giai đoạn 1: Gửi yêu cầu truy vấn

- **Bước 1:** Người dùng truy cập màn hình **History**, sau đó có thể nhập từ khóa tìm kiếm, lựa chọn thiết bị, số lượng bản ghi hiển thị hoặc tiêu chí sắp xếp.
- **Bước 2:** Frontend tổng hợp các điều kiện người dùng đã lựa chọn thành các **query parameter** và gửi HTTP Request **GET /api/actions** tới Backend.

Ví dụ:

GET /api/actions?deviceId=1&search=...&sort=DESC&page=1&limit=10

Giai đoạn 2: Truy xuất và xử lý dữ liệu

- **Bước 3:** Backend tiếp nhận Request, kiểm tra các query parameter và thực hiện truy vấn dữ liệu trong bảng **History**, đồng thời liên kết với bảng **Devices** để lấy thông tin thiết bị tương ứng.
- **Bước 4:** Database thực hiện truy vấn theo các điều kiện tìm kiếm, lọc, sắp xếp và phân trang, sau đó trả danh sách bản ghi phù hợp về Backend.
- **Bước 5:** Backend tiếp nhận kết quả và xử lý dữ liệu về định dạng phù hợp trước khi trả về Frontend.

Giai đoạn 3: Phản hồi và hiển thị

- **Bước 6:** Backend trả Response **200 OK** về Frontend, bao gồm danh sách lịch sử hoạt động và thông tin phân trang.
- **Bước 7:** Frontend nhận dữ liệu và hiển thị lên bảng **Action History** với các thông tin gồm **ID, thời gian, thiết bị, hành động và trạng thái thực hiện**.
- **Bước 8:** Người dùng có thể tiếp tục thay đổi điều kiện tìm kiếm, thiết bị, cách sắp xếp hoặc chuyển trang để thực hiện truy vấn mới.

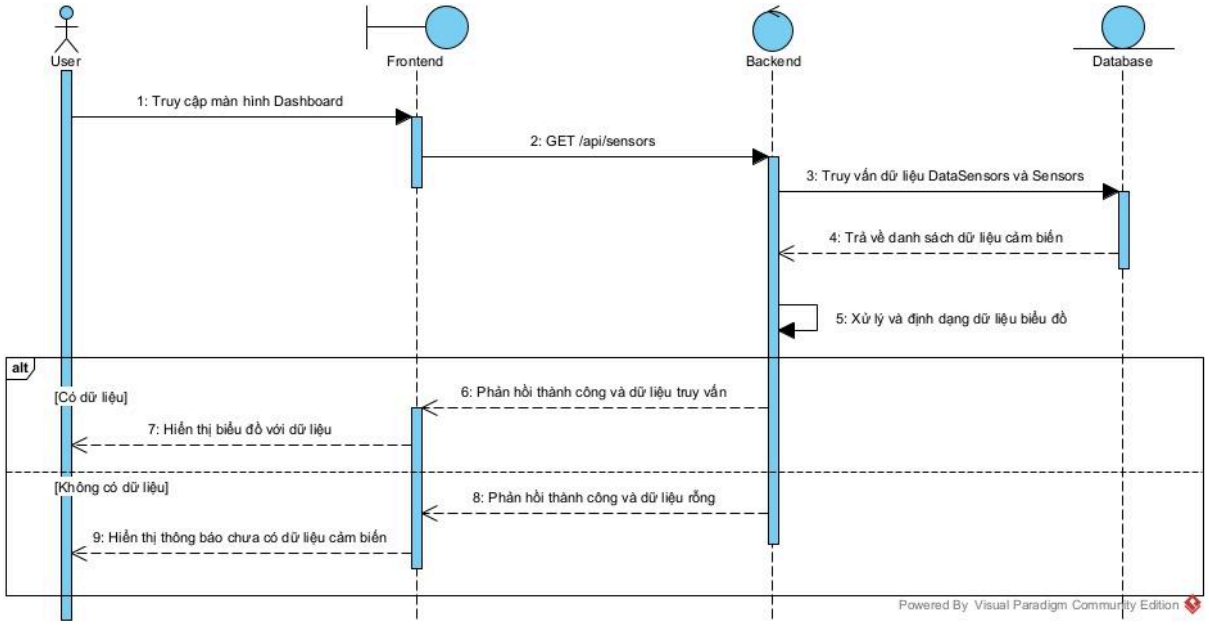
3.3.5. Biểu đồ dữ liệu cảm biến

Tên chức năng	Xem dữ liệu cảm biến
Mô tả	Cho phép người dùng theo dõi sự thay đổi của các thông số nhiệt độ, độ ẩm và cường độ ánh sáng trong nhà dưới dạng biểu đồ đường trên Dashboard
Đường dẫn	Web → Data Sensor → Khu vực biểu đồ dữ liệu cảm biến
Tác nhân	Người dùng đã đăng nhập vào hệ thống.
Luồng cơ bản	**Bước 1:** Người dùng truy cập màn hình **Dashboard**. **Bước 2:** Frontend gửi HTTP Request **GET /api/sensors** tới Backend để lấy dữ liệu cảm biến phục vụ việc hiển thị biểu đồ. **Bước 3:** Backend tiếp nhận yêu cầu và thực hiện truy vấn

	dữ liệu trong bảng **DataSensors**, đồng thời xác định loại cảm biến tương ứng từ bảng **Sensors**. **Bước 4:** Database thực hiện truy vấn và trả danh sách dữ liệu nhiệt độ, độ ẩm và ánh sáng về Backend. **Bước 5:** Backend xử lý dữ liệu, sắp xếp các bản ghi theo thời gian và trả kết quả về Frontend với mã trạng thái **200 OK**. **Bước 6:** Frontend nhận dữ liệu và hiển thị các giá trị dưới dạng biểu đồ đường trên Dashboard. Dữ liệu nhiệt độ và độ ẩm được thể hiện trên cùng một biểu đồ, trong khi dữ liệu ánh sáng được hiển thị trên biểu đồ riêng.
Luồng ngoại lệ	**Trường hợp 1 – Không có dữ liệu cảm biến:** 1. Database không có bản ghi cảm biến phù hợp để hiển thị. 2. Backend trả về danh sách dữ liệu rỗng. 3. Frontend hiển thị biểu đồ trống hoặc thông báo **"Chưa có dữ liệu cảm biến"**. **Trường hợp 2 – Lỗi truy vấn Database:** 1. Xảy ra lỗi trong quá trình Backend truy xuất dữ liệu từ Database. 2. Backend xử lý ngoại lệ và trả thông báo lỗi về Frontend. 3. Frontend hiển thị cảnh báo **"Không thể tải dữ liệu biểu đồ"** cho người dùng.
Điều kiện trước	Người dùng đã đăng nhập vào hệ thống. Frontend, Backend và Database đang hoạt động.

	Hệ thống đã có dữ liệu nhiệt độ, độ ẩm và ánh sáng được lưu trong bảng **DataSensors**.
Điều kiện sau	**Trường hợp thành công:** Biểu đồ nhiệt độ, độ ẩm và ánh sáng được hiển thị trên Dashboard dựa trên dữ liệu cảm biến đã lưu trong hệ thống. **Trường hợp không có dữ liệu hoặc xảy ra lỗi:** Frontend hiển thị biểu đồ trống hoặc thông báo lỗi tương ứng cho người dùng.

Sequence diagram:



Hình 11. Sequence diagram chức năng Biểu đồ dữ liệu cảm biến

Mô tả luồng chức năng:

Giai đoạn 1: Gửi yêu cầu lấy dữ liệu biểu đồ

- **Bước 1:** Người dùng truy cập màn hình **Dashboard**, nơi hiển thị các biểu đồ theo dõi nhiệt độ, độ ẩm và ánh sáng.

- **Bước 2:** Frontend gửi HTTP Request **GET /api/sensors** tới Backend để yêu cầu lấy dữ liệu cảm biến phục vụ cho việc hiển thị biểu đồ.

Giai đoạn 2: Truy xuất và xử lý dữ liệu

- **Bước 3:** Backend tiếp nhận Request và thực hiện truy vấn dữ liệu cảm biến đã được lưu trong Database.
- **Bước 4:** Database tìm kiếm và trả về danh sách các bản ghi cảm biến theo thời gian cho Backend.
- **Bước 5:** Backend tiếp nhận kết quả, phân loại dữ liệu theo từng loại cảm biến và xử lý dữ liệu về cấu trúc phù hợp để Frontend có thể sử dụng để vẽ biểu đồ.

Giai đoạn 3: Phản hồi và hiển thị

- **Bước 6:** Backend trả Response **200 OK** về Frontend, kèm theo dữ liệu nhiệt độ, độ ẩm và ánh sáng đã được xử lý.
- **Bước 7:** Frontend tiếp nhận dữ liệu và hiển thị lên các biểu đồ trên Dashboard.

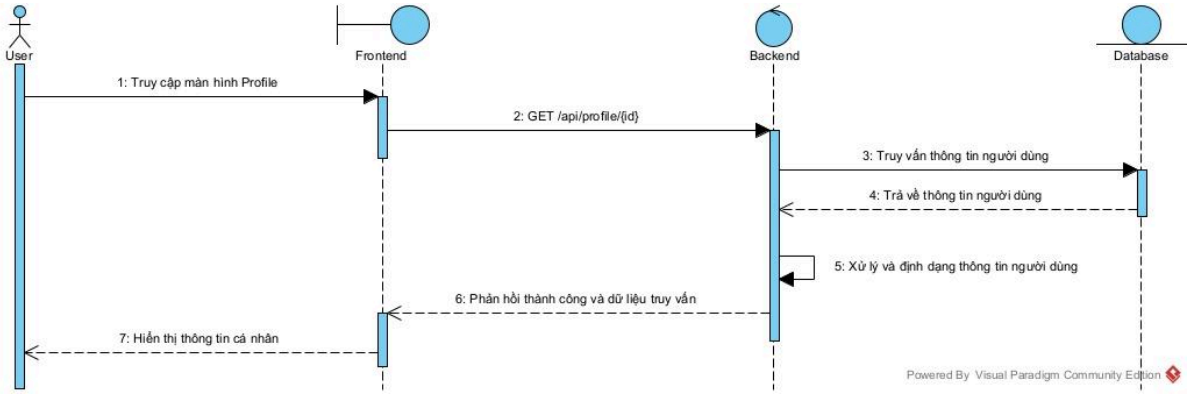
3.3.6. Xem thông tin cá nhân người dùng

Tên chức năng	Xem thông tin cá nhân người dùng
Mô tả	Cho phép người dùng xem các thông tin cá nhân được lưu trong hệ thống, bao gồm họ tên, mã sinh viên, email, chuyên ngành, trường học và các tài nguyên liên quan đến dự án như tài liệu hệ thống, API, GitHub và Figma.
Đường dẫn	Web → Profile
Tác nhân	Người dùng đã đăng nhập vào hệ thống.
Luồng cơ bản	**Bước 1:** Người dùng chọn mục **Profile** trên thanh điều hướng của giao diện Web.

	Bước 2: Frontend gửi HTTP Request **GET /api/profile/{id}** tới Backend để lấy thông tin cá nhân của người dùng tương ứng. **Bước 3:** Backend tiếp nhận yêu cầu, kiểm tra thông tin người dùng và thực hiện truy vấn dữ liệu hồ sơ trong Database. **Bước 4:** Database tìm kiếm và trả thông tin người dùng cùng các dữ liệu liên quan về Backend. **Bước 5:** Backend xử lý dữ liệu và trả Response **200 OK** về Frontend. **Bước 6:** Frontend nhận dữ liệu và hiển thị thông tin cá nhân cùng các tài nguyên liên quan như **System Documentation, API Reference, GitHub Repository và Figma Design** trên màn hình Profile.
Luồng ngoại lệ	● **Trường hợp 1 – Không tìm thấy thông tin người dùng:** Backend không tìm thấy người dùng tương ứng với ID được yêu cầu và trả thông báo lỗi về Frontend. Frontend hiển thị thông báo không tìm thấy thông tin người dùng. ● **Trường hợp 2 – Lỗi truy vấn Database:** Nếu xảy ra lỗi trong quá trình truy xuất dữ liệu, Backend xử lý ngoại lệ và trả thông báo lỗi. Frontend hiển thị cảnh báo không thể tải thông tin cá nhân tại thời điểm hiện tại. .
Điều kiện trước	Người dùng đã đăng nhập vào hệ thống. Frontend, Backend và Database đang hoạt động. Thông tin người dùng đã được lưu trong hệ thống.

Điều kiện sau	**Trường hợp thành công:** Thông tin cá nhân và các tài nguyên liên quan được hiển thị đầy đủ trên màn hình Profile. **Trường hợp xảy ra lỗi:** Frontend hiển thị thông báo tương ứng và không hiển thị dữ liệu không hợp lệ.

Sequence diagram:



Hình 13. Sequence diagram chức năng xem thông tin cá nhân

Mô tả luồng chức năng:

Giai đoạn 1: Gửi yêu cầu lấy thông tin cá nhân

- **Bước 1:** Người dùng truy cập vào màn hình **Profile** từ thanh điều hướng của hệ thống.
- **Bước 2:** Frontend gửi HTTP Request **GET /api/profile/{id}** tới Backend để yêu cầu lấy thông tin chi tiết của người dùng theo mã định danh tương ứng.

Giai đoạn 2: Truy xuất và xử lý dữ liệu

- **Bước 3:** Backend tiếp nhận Request, kiểm tra thông tin người dùng và thực hiện truy vấn dữ liệu hồ sơ trong Database.
- **Bước 4:** Database tìm kiếm và trả về thông tin người dùng cùng các dữ liệu liên quan cho Backend.
- **Bước 5:** Backend tiếp nhận kết quả, xử lý và định dạng dữ liệu về cấu trúc phù hợp trước khi trả về Frontend.

Giai đoạn 3: Phản hồi và hiển thị

- **Bước 6:** Backend trả Response **200 OK** về Frontend, kèm theo thông tin cá nhân và các tài nguyên liên quan của người dùng.
- **Bước 7:** Frontend tiếp nhận dữ liệu và hiển thị lên màn hình Profile

Chương 4: Kết quả thực hiện

4.1. Kết quả triển khai hệ thống

Hệ thống Smart Home đã được tích hợp giữa phần cứng ESP32, MQTT Broker, Backend Spring Boot, cơ sở dữ liệu MySQL và Frontend React. Người dùng có thể giám sát dữ liệu môi trường và điều khiển các thiết bị mô phỏng từ giao diện Web

4.1.1. Triển khai phần cứng và chương trình ESP32

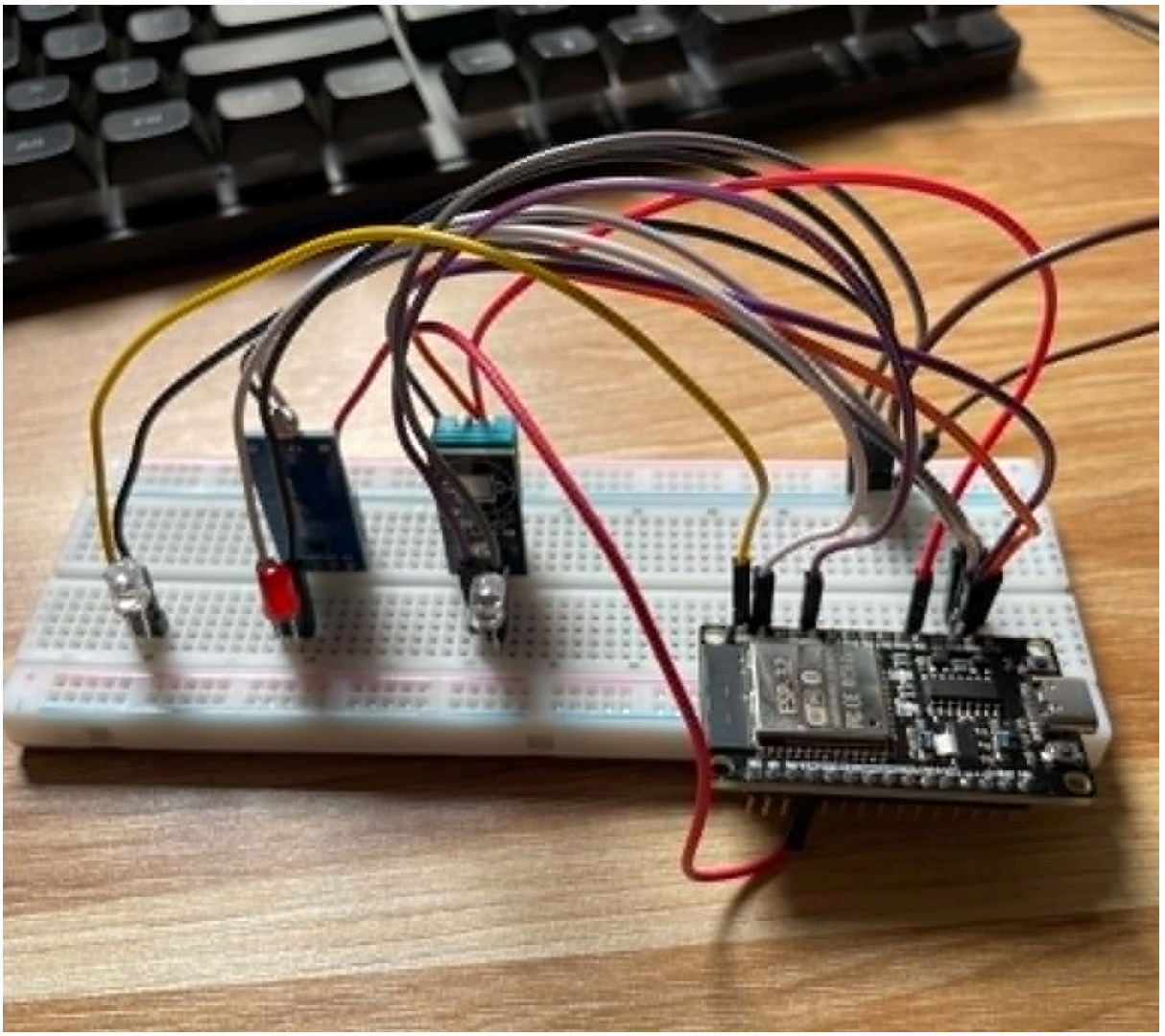
Mô hình sử dụng ESP32 DevKit V1, cảm biến DHT11, module quang trở LDR và ba LED mô phỏng đèn, quạt, điều hòa. Các linh kiện được kết nối trên breadboard; ESP32 được cấp nguồn qua USB.

Thành phần	Chân kết nối	Chức năng
DHT11	GPIO 4	Đo nhiệt độ và độ ẩm
LDR – chân AO	GPIO 34	Đọc tín hiệu ánh sáng
LED 1	GPIO 23	Mô phỏng đèn
LED 2	GPIO 22	Mô phỏng quạt
LED 3	GPIO 21	Mô phỏng điều hòa

Chương trình ESP32 được xây dựng bằng Arduino IDE. Sau khi khởi động, thiết bị kết nối Wi-Fi và MQTT Broker, đăng ký nhận lệnh điều khiển và gửi yêu cầu khôi phục trạng thái thiết bị.

Trong quá trình hoạt động, ESP32 đọc cảm biến theo chu kỳ khoảng hai giây. Dữ liệu hợp lệ được đóng gói thành JSON và gửi lên topic sensor/data. Giá trị ánh sáng được quy đổi thành lux ước tính từ tín hiệu analog của LDR.

Khi nhận lệnh điều khiển, ESP32 xác định thiết bị, thiết lập mức logic GPIO và gửi phản hồi kèm historyId. Việc đọc lại GPIO chỉ kiểm tra mức logic của chân điều khiển, chưa xác nhận được LED có thực sự phát sáng hay không.



Hình 4.1. Mô hình phần cứng của hệ thống Smart Home.

4.1.2. Triển khai MQTT Broker

Mosquitto được sử dụng làm MQTT Broker, hoạt động trên cổng 1884 và yêu cầu xác thực bằng tên người dùng, mật khẩu. Broker chuyển tiếp dữ liệu giữa ESP32 và Backend theo cơ chế Publish/Subscribe.

Các topic được sử dụng gồm:

Topic	Chức năng
sensor/data	Truyền dữ liệu cảm biến từ ESP32 tới Backend
device/control/{id}	Gửi lệnh điều khiển tới thiết bị
device/status/{id}	Phản hồi kết quả điều khiển
device/status/connection	Thông báo trạng thái kết nối ESP32
device/status/restore	Yêu cầu khôi phục trạng thái thiết bị
device/control/restore	Trả trạng thái cần khôi phục cho ESP32

ESP32 đăng ký Last Will với nội dung OFFLINE. Khi phát hiện kết nối bị mất bất thường, Broker gửi thông báo này tới Backend để cập nhật lên giao diện.

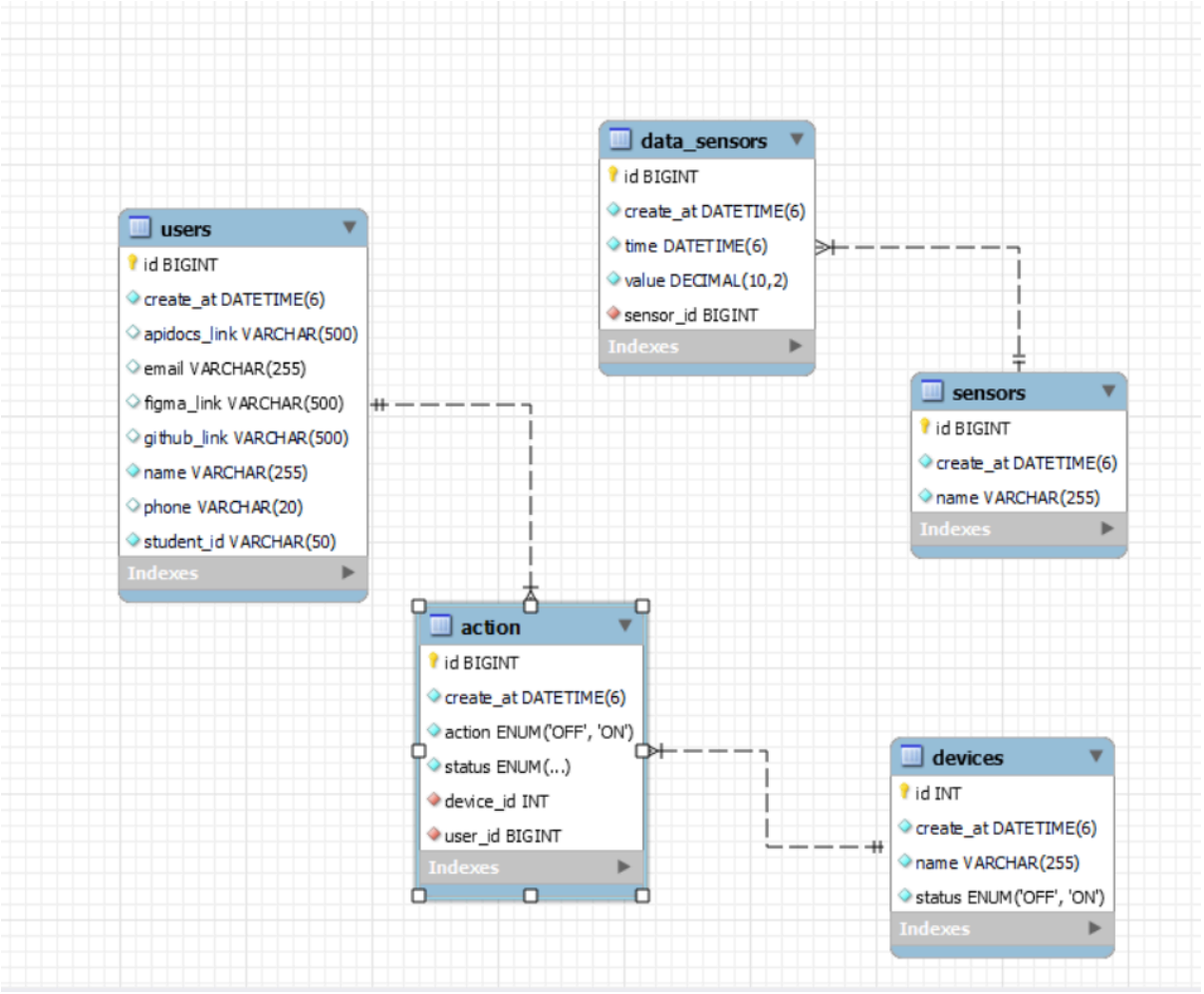
4.1.3. Triển khai cơ sở dữ liệu

Hệ thống sử dụng MySQL để lưu trữ dữ liệu trong năm bảng chính:

Bảng	Nội dung
users	Thông tin người dùng và liên kết tài nguyên
sensors	Danh mục cảm biến
data_sensors	Giá trị cảm biến theo thời gian

devices	Thông tin thiết bị và trạng thái ON/OFF
action	Lịch sử điều khiển và kết quả thực hiện

Dữ liệu cảm biến được lưu để phục vụ hiển thị, tìm kiếm và vẽ biểu đồ. Lịch sử điều khiển ghi nhận riêng hành động ON/OFF và kết quả PENDING, SUCCESS, ERROR, TIMEOUT.



Hình 4.2. Cơ sở dữ liệu của hệ thống.

4.1.4. Triển khai Backend Spring Boot

Backend được xây dựng bằng Java Spring Boot, tổ chức theo các lớp Controller, Service và Repository. Các chức năng chính gồm:

- Nhận và kiểm tra dữ liệu cảm biến từ MQTT, lưu dữ liệu vào MySQL.
- Cung cấp REST API phục vụ Dashboard, tra cứu dữ liệu, lịch sử điều khiển và hồ sơ.
- Đánh giá số đo theo ngưỡng cảnh báo đã cấu hình.
- Tiếp nhận lệnh điều khiển, tạo lịch sử PENDING và gửi lệnh tới ESP32.
- Xử lý phản hồi để cập nhật SUCCESS hoặc ERROR.
- Chuyển lệnh quá hạn sang TIMEOUT bằng tác vụ định kỳ.
- Gửi dữ liệu và kết quả điều khiển qua WebSocket sau khi transaction commit thành công.
- Thông báo trạng thái kết nối và trả trạng thái trong DB cho ESP32 khôi phục sau khi kết nối lại.

Thời gian chờ phản hồi điều khiển được cấu hình là 30 giây. Scheduler quét định kỳ khoảng 5 giây nên thời điểm ghi nhận TIMEOUT phụ thuộc vào lần quét sau khi lệnh quá hạn.

4.1.5. Triển khai Frontend React

Frontend được xây dựng bằng React và Vite. REST API được sử dụng để lấy dữ liệu ban đầu, tìm kiếm và gửi yêu cầu điều khiển. Kết nối STOMP qua WebSocket giúp Dashboard nhận dữ liệu mới mà không cần tải lại trang.

Giao diện xử lý các trạng thái đang tải, dữ liệu rỗng, lỗi kết nối và chờ kết quả điều khiển. Khi nhận kết quả từ Backend, công tắc và thông báo được cập nhật theo lệnh tương ứng.

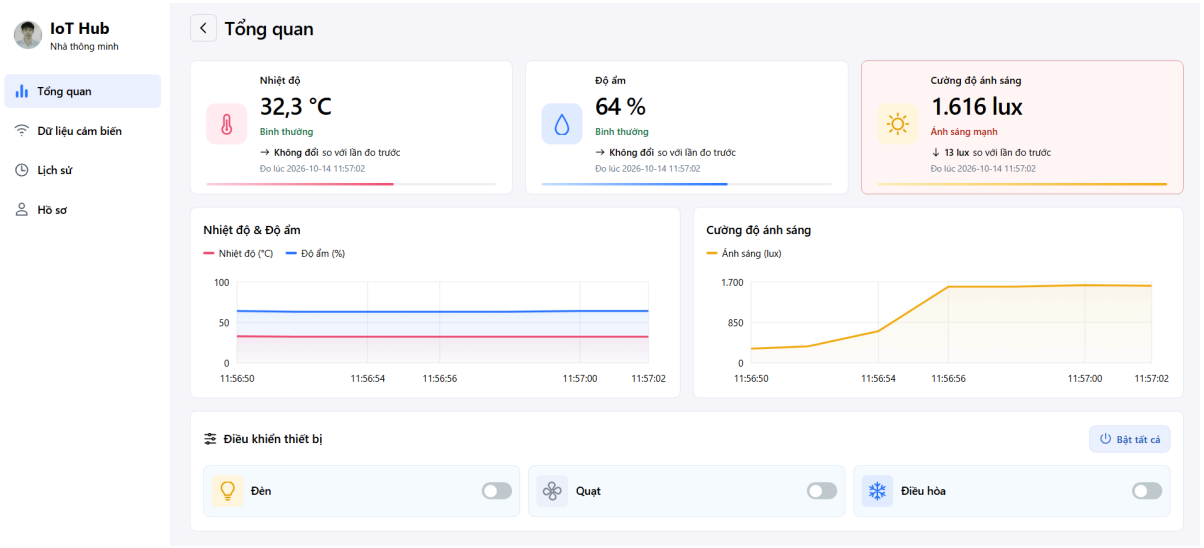
4.2. Kết quả giao diện và chức năng

4.2.1. Màn hình Dashboard

Dashboard hiển thị ba thông số môi trường gồm nhiệt độ, độ ẩm và ánh sáng. Mỗi thẻ cảm biến thể hiện số đo, đơn vị, trạng thái cảnh báo và xu hướng tăng hoặc giảm so với lần đo trước.

Biểu đồ thể hiện sự thay đổi của các thông số theo thời gian. Khi mở trang, dữ liệu lịch sử được tải qua REST API; các điểm đo mới được bổ sung qua WebSocket.

Khu vực điều khiển cho phép bật/tắt từng thiết bị hoặc thao tác với tất cả thiết bị. Khi lệnh đang chờ phản hồi, giao diện hiển thị trạng thái đang xử lý. Kết quả thành công, lỗi hoặc hết thời gian chờ được cập nhật khi nhận thông báo từ Backend.

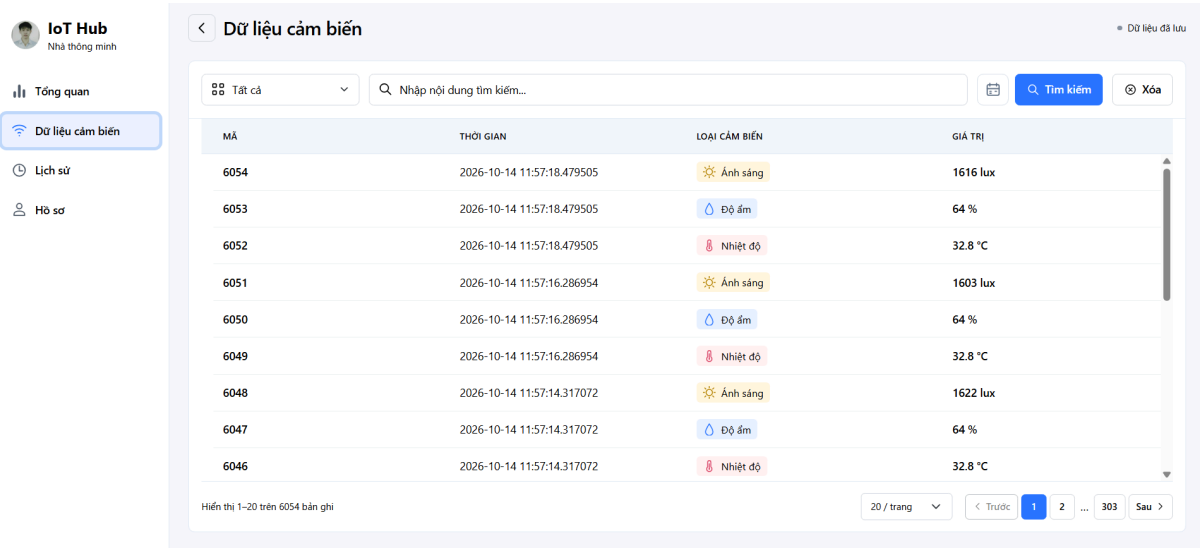


Hình 4.3. Giao diện Dashboard.

4.2.2. Màn hình DataSensor

DataSensor hiển thị danh sách dữ liệu cảm biến đã được lưu, gồm mã bản ghi, thời gian, loại cảm biến và giá trị đo.

Người dùng có thể tìm kiếm theo trường dữ liệu, lọc khoảng ngày, thay đổi số bản ghi trên mỗi trang và chuyển trang. Khi không chọn khoảng ngày, hệ thống truy vấn theo các điều kiện còn lại.

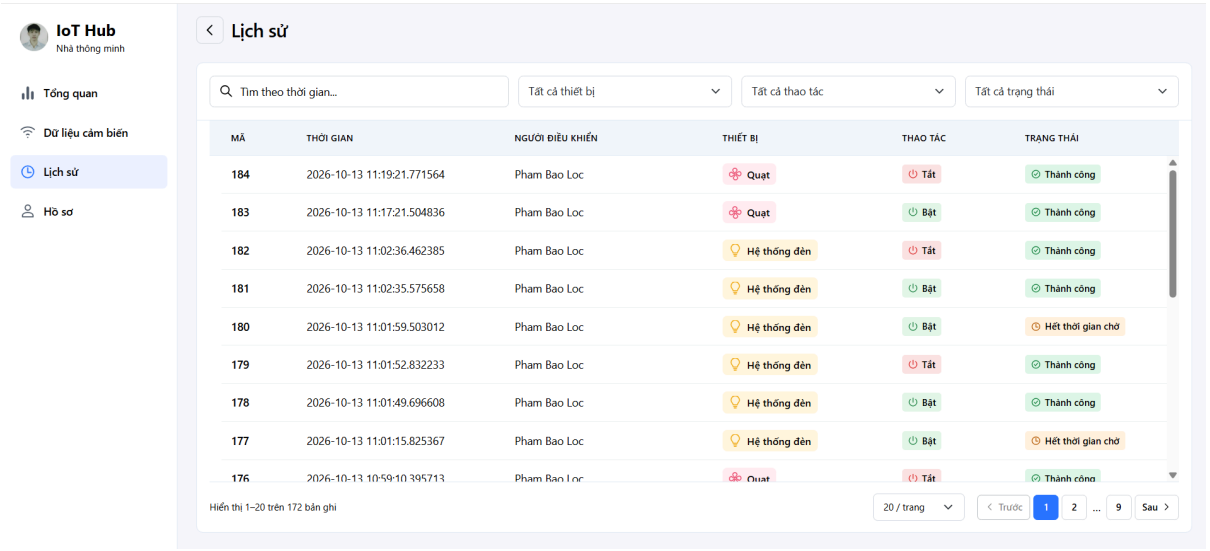


Hình 4.4. Giao diện tra cứu dữ liệu cảm biến.

4.2.3. Màn hình History

History hiển thị lịch sử điều khiển thiết bị và hỗ trợ lọc theo thời gian, thiết bị, hành động và kết quả thực hiện.

Mỗi bản ghi phân biệt hành động người dùng yêu cầu với kết quả xử lý. Ví dụ, một lệnh ON có thể nhận kết quả SUCCESS khi ESP32 phản hồi thành công hoặc TIMEOUT nếu quá thời gian chờ.

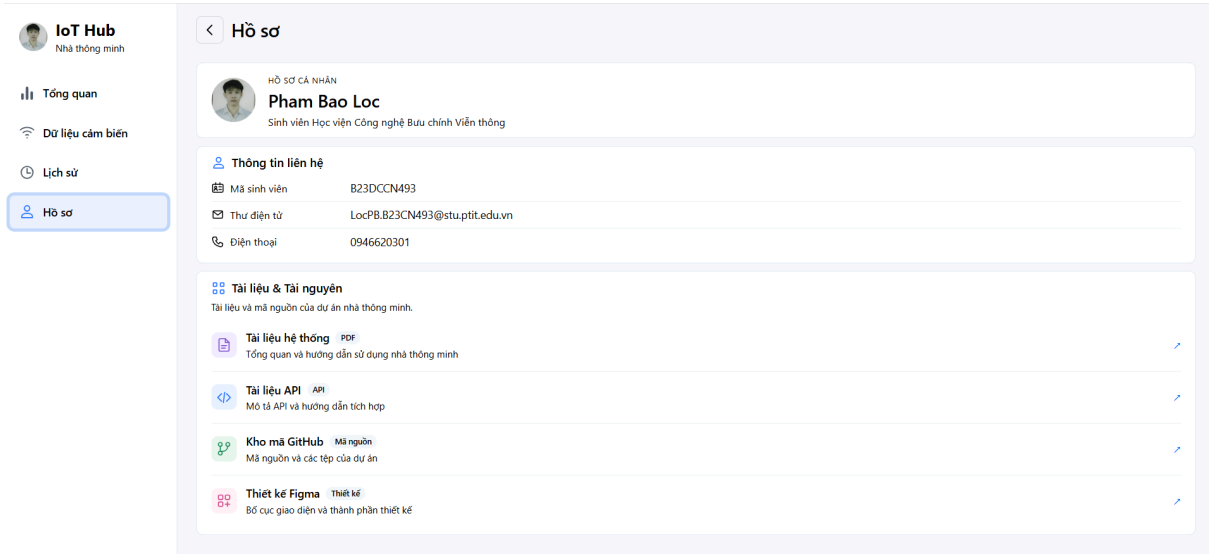


Hình 4.5. Giao diện lịch sử điều khiển thiết bị.

4.2.4. Màn hình Profile

Profile hiển thị thông tin người thực hiện và các tài nguyên của dự án. Dữ liệu hồ sơ được lấy từ Backend với người dùng có mã định danh 1.

Các liên kết tài nguyên hỗ trợ truy cập mã nguồn, thiết kế giao diện, tài liệu API và báo cáo. Khi không tải được hồ sơ, giao diện hiển thị thông báo và nút thử kết nối lại.



Hình 4.6. Giao diện hồ sơ người dùng.

4.3. Kiểm thử hệ thống

Các API được kiểm tra bằng Postman thông qua việc gửi request tới Backend và quan sát mã trạng thái HTTP, cấu trúc response và dữ liệu trả về. Những hình minh họa dưới đây cần được chụp từ kết quả chạy thực tế.

Địa chỉ Backend trong môi trường kiểm thử:

http://localhost:8080

4.3.1. API lấy dữ liệu cảm biến mới nhất

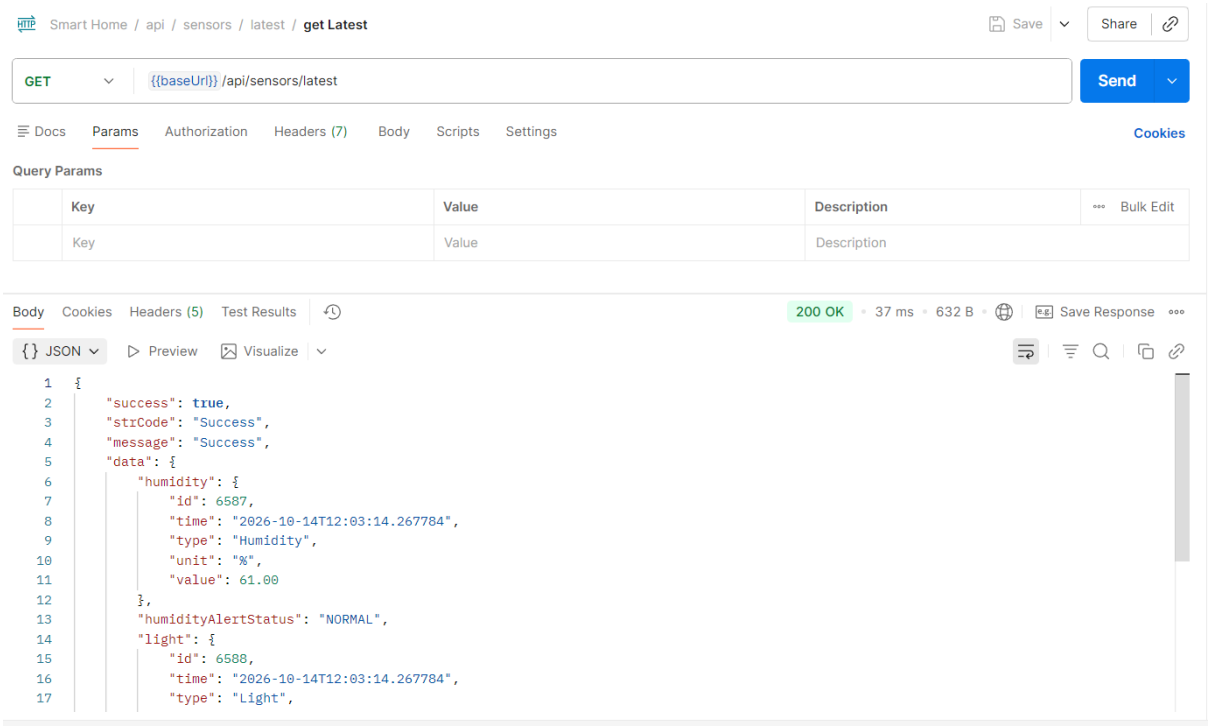
Mục tiêu: Kiểm tra API cung cấp số đo mới nhất để hiển thị trên Dashboard.

Request:

GET /api/sensors/latest

Tham số: Không có.

Nội dung đối chiếu: HTTP 200 OK; response chứa dữ liệu nhiệt độ, độ ẩm, ánh sáng và trạng thái cảnh báo tương ứng khi đã có dữ liệu trong hệ thống.



Hình 4.8. Kiểm thử API lấy dữ liệu cảm biến mới nhất.

4.3.2. API lấy dữ liệu biểu đồ

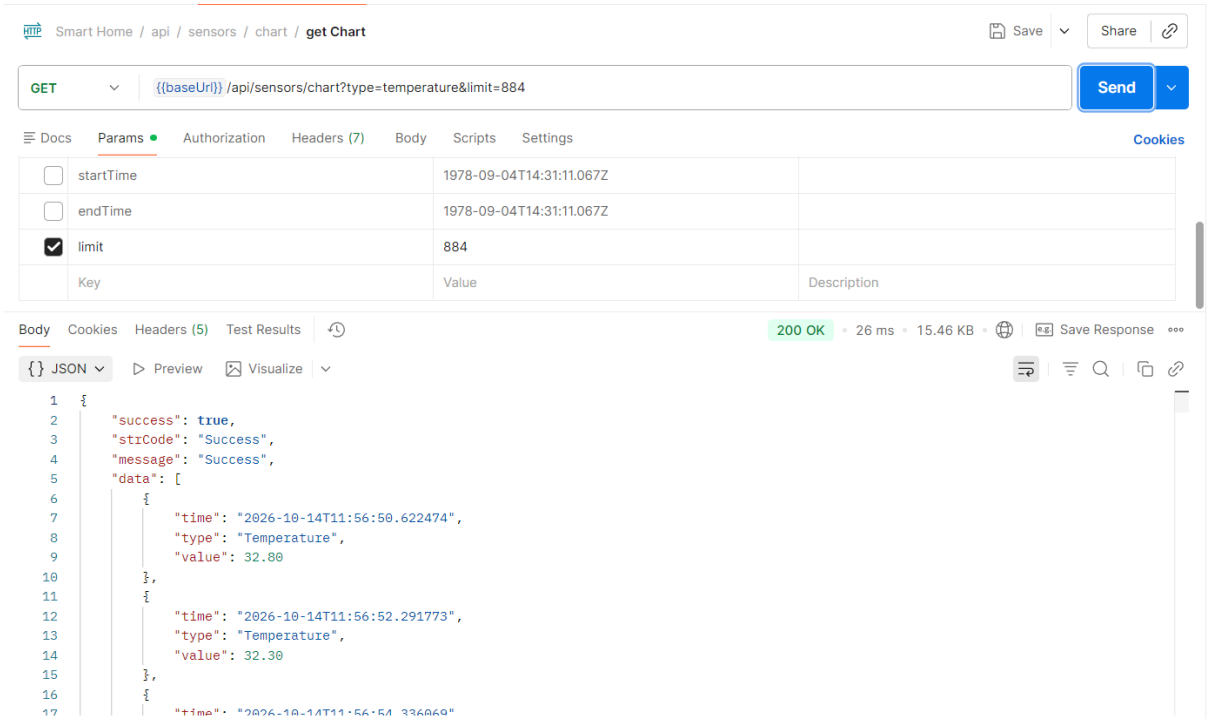
Mục tiêu: Kiểm tra dữ liệu lịch sử dùng để vẽ biểu đồ cho từng loại cảm biến.

Request:

GET /api/sensors/chart

Tham số: Chọn loại cảm biến và giới hạn số điểm theo các trường của request đã import vào Postman. Có thể bổ sung khoảng thời gian để kiểm tra truy vấn.

Nội dung đối chiếu: HTTP 200 OK; các điểm dữ liệu thuộc đúng loại cảm biến, có thời gian và giá trị đo, được sắp xếp theo thời gian để hiển thị trên biểu đồ.



Hình 4.9. Kiểm thử API lấy dữ liệu biểu đồ.

4.3.3. API tra cứu dữ liệu cảm biến

Mục tiêu: Kiểm tra khả năng lọc dữ liệu theo loại cảm biến và phân trang.

Request:

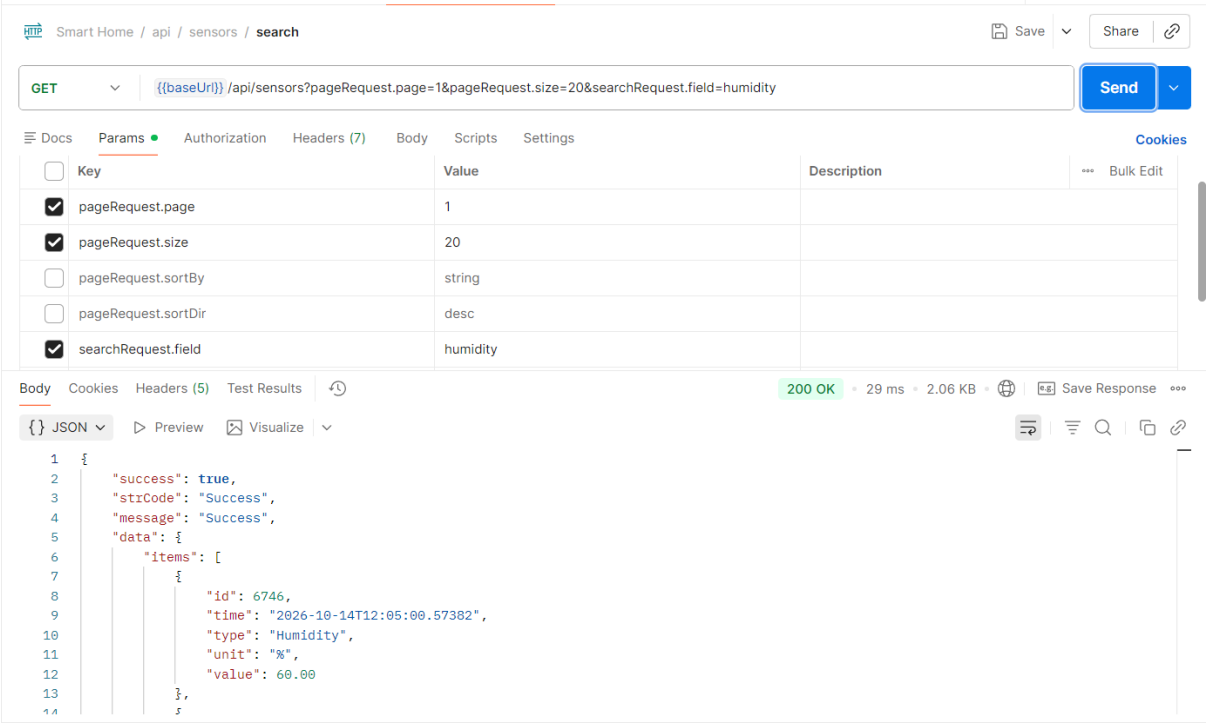
GET
/api/sensors?pageRequest.page=1&pageRequest.size=20&searchRequest.field=humidity

Tham số:

Tham số	Giá trị	Ý nghĩa
pageRequest.page	1	Lấy trang đầu tiên
pageRequest.size	20	Tối đa 20 bản ghi trên một trang

searchRequest.field	humidity	Lọc dữ liệu độ ẩm

Nội dung đối chiếu: HTTP 200 OK; data.items chứa dữ liệu độ ẩm và tối đa 20 phần tử. Response có các trường page, size, totalElements và totalPages.



Hình 4.10. Kiểm thử API tra cứu dữ liệu cảm biến.

4.3.4. API lấy danh sách thiết bị

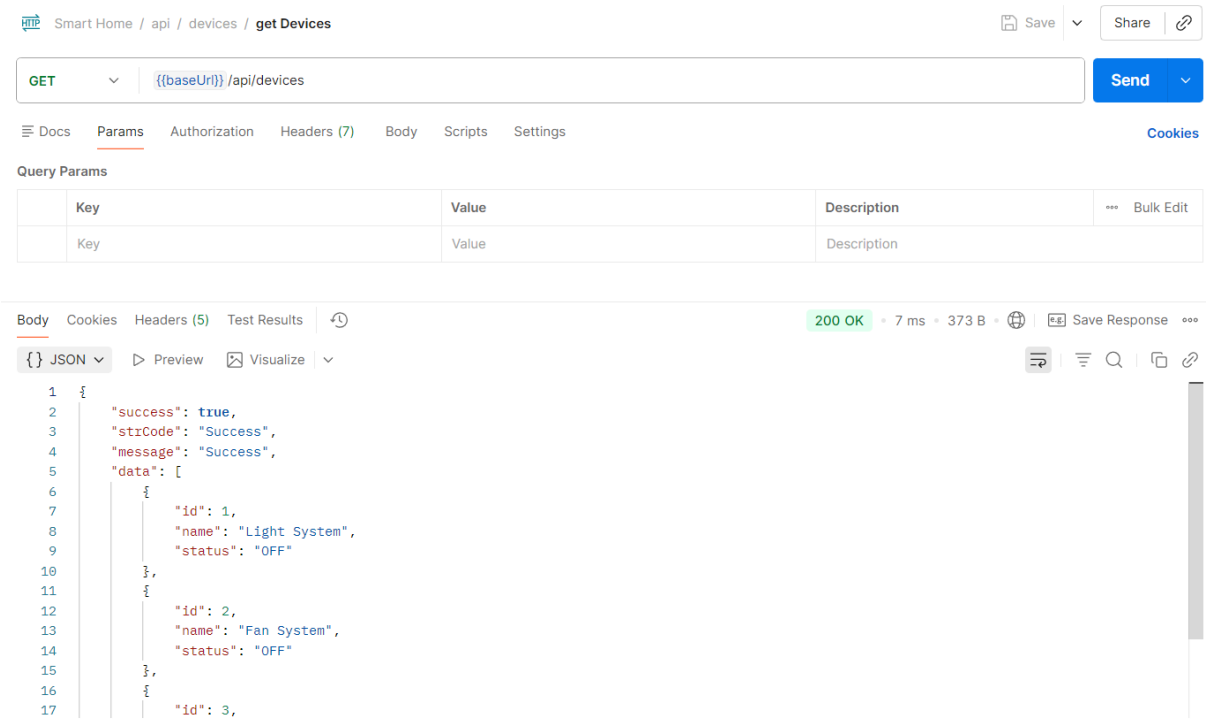
Mục tiêu: Kiểm tra dữ liệu thiết bị dùng để hiển thị các công tắc trên Dashboard.

Request:

GET /api/devices

Tham số: Không có.

Nội dung đối chiếu: HTTP 200 OK; response trả danh sách thiết bị gồm mã định danh, tên và trạng thái ON/OFF đang được lưu trong cơ sở dữ liệu.



Hình 4.11. Kiểm thử API lấy danh sách thiết bị.

4.3.5. API điều khiển thiết bị

Mục tiêu: Kiểm tra khả năng tiếp nhận và gửi lệnh điều khiển tới ESP32.

Request:

POST /api/devices/1/control?action=ON

Tham số:

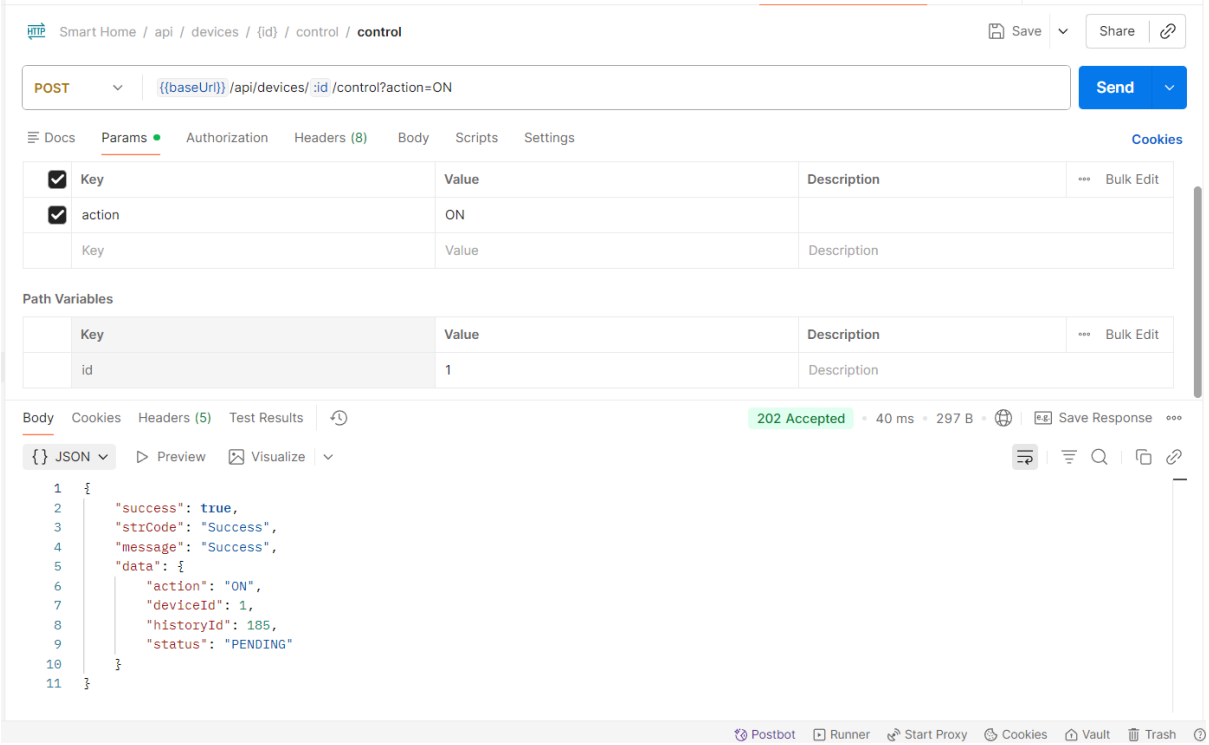
Tham số	Giá trị	Ý nghĩa
id	1	Thiết bị cần điều khiển
action	ON	Yêu cầu bật thiết bị

Body: Để trống.

Trước khi kiểm tra, thiết bị cần ở trạng thái OFF, không có lệnh PENDING và các thành phần liên quan đang hoạt động.

Nội dung đối chiếu: API trả HTTP 202 Accepted cùng thông tin lệnh, trong đó có historyId. ESP32 nhận lệnh, thay đổi GPIO và gửi phản hồi. Khi xử lý phản hồi thành công, Backend cập nhật lịch sử thành SUCCESS và trạng thái thiết bị thành ON.

Response HTTP ban đầu thể hiện việc tiếp nhận lệnh; kết quả thực hiện cuối cùng được đối chiếu qua lịch sử điều khiển, DB hoặc giao diện.



Hình 4.12. Kiểm thử API điều khiển thiết bị.

4.3.6. API tra cứu lịch sử điều khiển

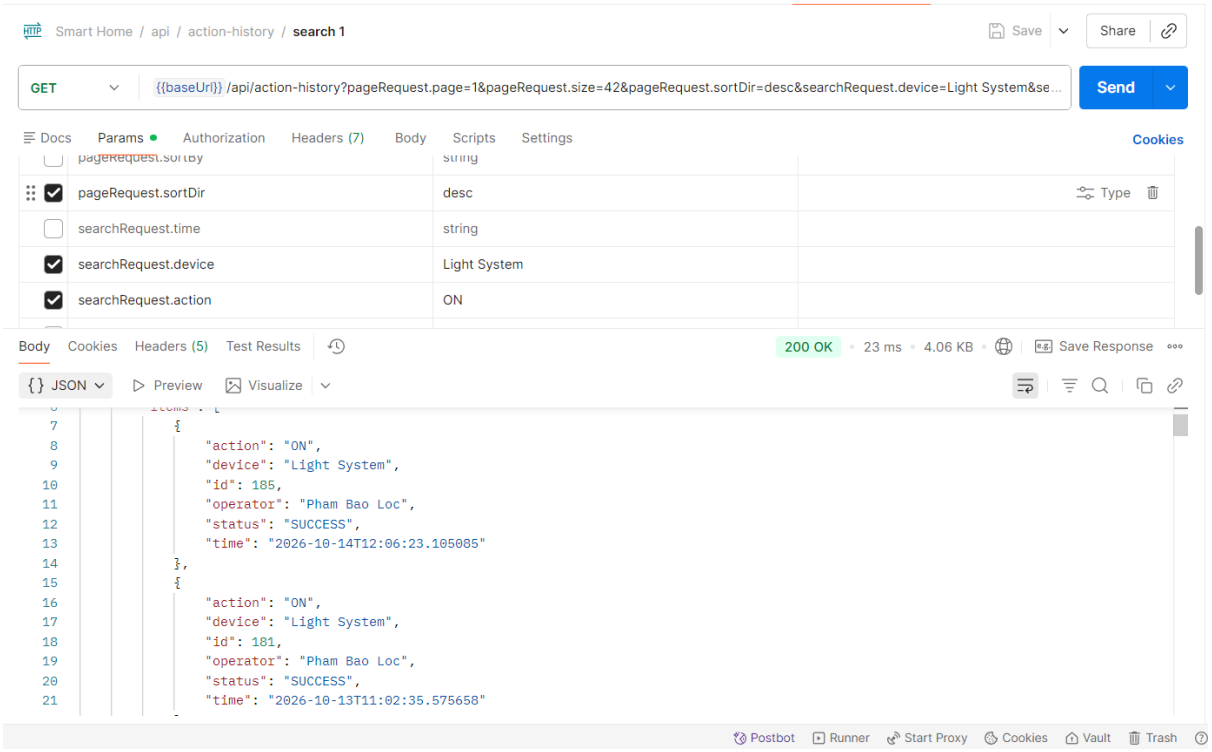
Mục tiêu: Kiểm tra khả năng truy vấn lịch sử và phân biệt hành động yêu cầu với kết quả thực hiện.

Request: Sử dụng request tra cứu lịch sử điều khiển trong collection Postman đã import từ Backend.

Tham số: Chọn trang, kích thước trang và các điều kiện lọc như thiết bị, hành động, trạng thái hoặc thời gian.

Nội dung đối chiếu: HTTP 200 OK; response chứa danh sách lịch sử đúng bộ lọc, kèm thông tin phân trang. Mỗi bản ghi thể hiện thiết bị, người thực hiện, thời gian, hành động và kết quả xử lý.

Có thể sử dụng historyId của lệnh vừa gửi ở mục 4.3.5 để đối chiếu kết quả.



Hình 4.13. Kiểm thử API tra cứu lịch sử điều khiển.

4.3.7. API lấy trạng thái kết nối

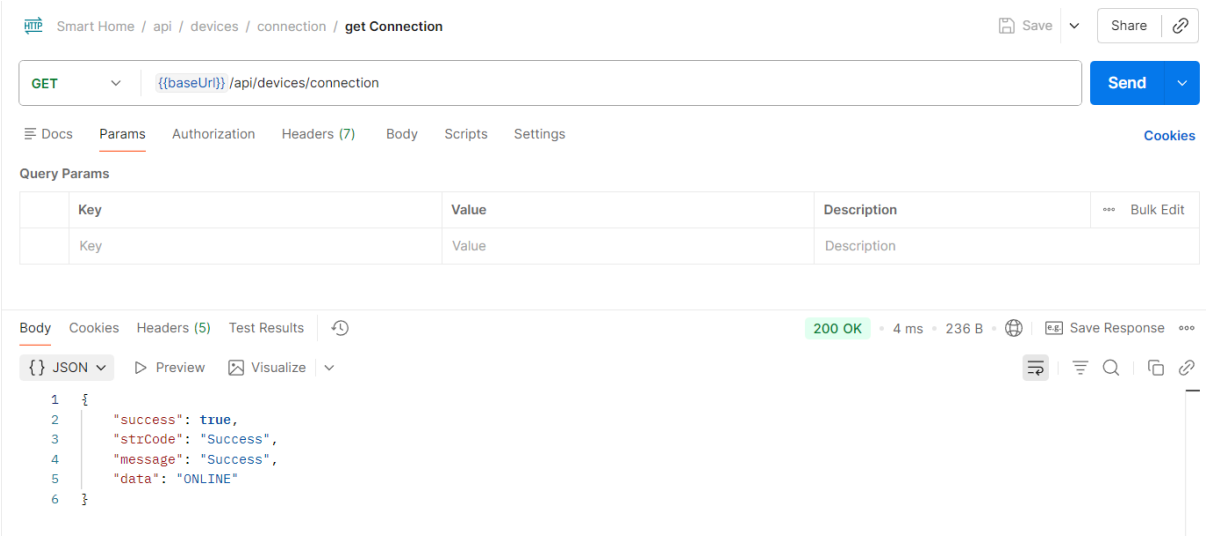
Mục tiêu: Kiểm tra trạng thái kết nối ESP32 mà Backend đang ghi nhận.

Request:

GET /api/devices/connection

Tham số: Không có.

Nội dung đối chiếu: HTTP 200 OK; dữ liệu trả về là ONLINE, OFFLINE hoặc UNKNOWN, tùy thông báo kết nối mà Backend đã nhận được.



Hình 4.14. Kiểm thử API lấy trạng thái kết nối ESP32.

4.3.8. API lấy thông tin hồ sơ

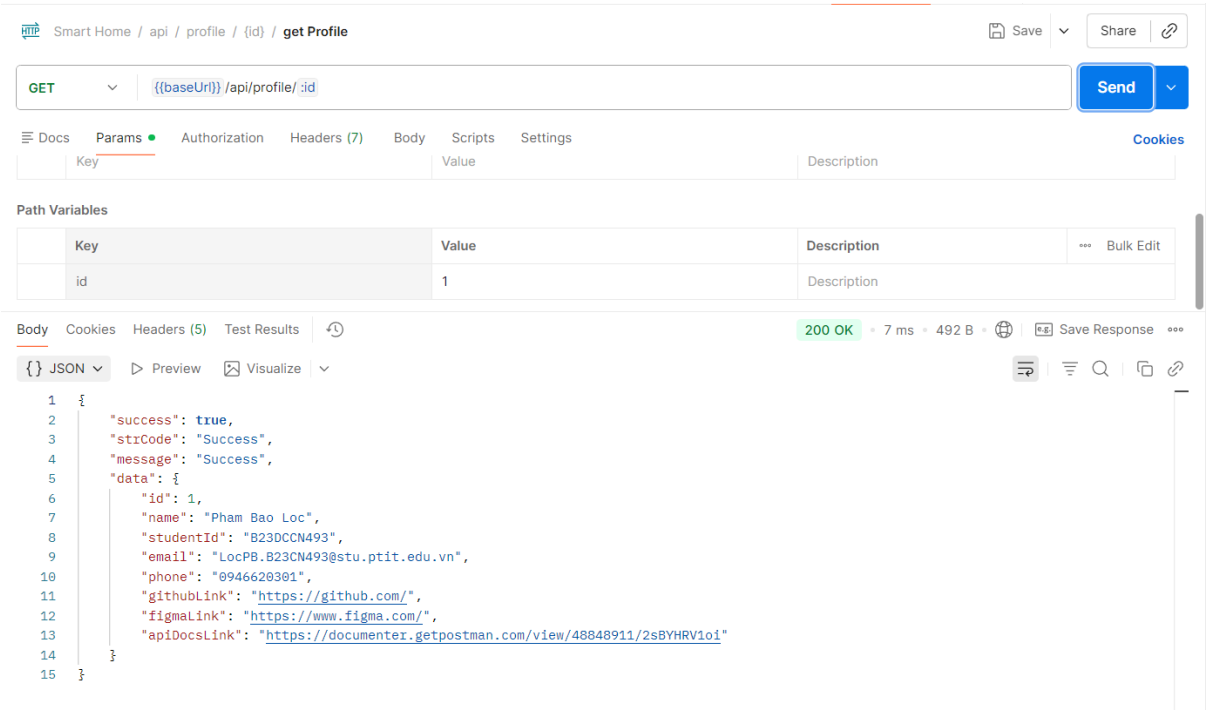
Mục tiêu: Kiểm tra dữ liệu người dùng được cung cấp cho trang Profile.

Request:

GET /api/profile/1

Tham số: Mã người dùng id = 1.

Nội dung đối chiếu: HTTP 200 OK; response chứa thông tin người dùng như họ tên, mã sinh viên, email, số điện thoại và các liên kết tài nguyên.



Hình 4.15. Kiểm thử API lấy thông tin hồ sơ.

4.4. Đánh giá kết quả

Hệ thống đã tích hợp được luồng giám sát từ ESP32 tới giao diện và luồng điều khiển từ giao diện tới thiết bị. Dữ liệu được lưu trong MySQL để phục vụ tra cứu, đồng thời được cập nhật qua WebSocket.

Nội dung	Kết quả triển khai
Giám sát môi trường	Thu thập nhiệt độ, độ ẩm và ánh sáng
Hiển thị thời gian thực	Cập nhật số đo, cảnh báo và biểu đồ
Điều khiển thiết bị	Bật/tắt ba LED mô phỏng
Theo dõi kết quả lệnh	Phân biệt PENDING, SUCCESS, ERROR, TIMEOUT

Tra cứu dữ liệu	Tìm kiếm, lọc và phân trang
Theo dõi kết nối	Hiển thị thông báo mất kết nối ESP32
Khôi phục trạng thái	Áp dụng trạng thái DB khi ESP32 kết nối lại
Hồ sơ người dùng	Hiển thị thông tin và liên kết tài nguyên

Mô hình đáp ứng các chức năng giám sát và điều khiển trong phạm vi đề tài. Những nội dung về độ chính xác cảm biến, bảo mật và khả năng mở rộng cần tiếp tục được hoàn thiện.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Đề tài đã xây dựng hệ thống Smart Home kết hợp ESP32, cảm biến, MQTT, Spring Boot, MySQL và React. Hệ thống thu thập dữ liệu môi trường, hiển thị số đo và biểu đồ theo thời gian thực, đồng thời hỗ trợ điều khiển thiết bị mô phỏng qua giao diện Web.

Các chức năng tìm kiếm dữ liệu, tra cứu lịch sử, cảnh báo vượt ngưỡng, thông báo mất kết nối và khôi phục trạng thái thiết bị đã được tích hợp. Việc tách trạng thái thiết bị khỏi kết quả lệnh giúp theo dõi quá trình điều khiển rõ ràng hơn.

Thông qua quá trình thực hiện, sinh viên đã vận dụng kiến thức về lập trình vi điều khiển, truyền thông MQTT, phát triển REST API, WebSocket và cơ sở dữ liệu để xây dựng một hệ thống IoT có khả năng hoạt động với phần cứng.

5.2. Hạn chế của hệ thống

- Hệ thống mới triển khai với một ESP32 và ba LED mô phỏng, chưa điều khiển thiết bị điện gia dụng thực tế.
- Giá trị ánh sáng là lux ước tính từ LDR, chưa được hiệu chuẩn; dữ liệu đo còn phụ thuộc đặc tính cảm biến.
- Phản hồi điều khiển dựa trên mức logic GPIO, chưa có cảm biến xác nhận hoạt động thực tế của thiết bị.

- Chưa triển khai đăng nhập và phân quyền; người thực hiện lệnh đang được xác định bằng cấu hình người dùng cố định.
- MQTT sử dụng tên người dùng và mật khẩu, chưa triển khai TLS; hệ thống chưa được hoàn thiện cho môi trường Internet công khai.
- Trạng thái kết nối ESP32 được lưu trong bộ nhớ Backend. Khi Backend ngừng hoạt động, một số thông báo kết nối có thể bị bỏ lỡ.
- Kiểm tra lệnh PENDING chưa bảo đảm loại bỏ hoàn toàn xung đột khi nhiều yêu cầu điều khiển đến cùng lúc.
- Hệ thống chưa được kiểm thử tải dài hạn hoặc triển khai với nhiều Backend chạy đồng thời.

5.3. Hướng phát triển

Trong các giai đoạn tiếp theo, hệ thống có thể được mở rộng theo các hướng:

- Bổ sung xác thực, phân quyền và xác định người thực hiện lệnh theo tài khoản đăng nhập.
- Triển khai HTTPS/WSS, TLS và phân quyền topic MQTT.
- Hoàn thiện cơ chế xử lý yêu cầu đồng thời và đồng bộ trạng thái khi các thành phần khởi động lại.
- Hiệu chuẩn cảm biến hoặc sử dụng cảm biến ánh sáng chuyên dụng.
- Thay LED bằng mạch chấp hành phù hợp, có cách ly và bảo vệ để điều khiển thiết bị thực tế.
- Mở rộng quản lý nhiều phòng, nhiều ESP32 và nhiều loại thiết bị.
- Bổ sung điều khiển theo lịch, quy tắc tự động và thông báo từ xa.
- Kiểm thử tải, tối ưu truy vấn và xây dựng chính sách lưu trữ dữ liệu lịch sử.

5.4. Kết luận

Đề tài đã hoàn thành mô hình giám sát môi trường và điều khiển thiết bị trong nhà thông minh. Các thành phần phần cứng và phần mềm được kết nối thành luồng hoạt động hai chiều, hỗ trợ theo dõi dữ liệu, xác nhận kết quả điều khiển và xử lý một số tình huống mất kết nối.

Kết quả thực hiện là nền tảng để tiếp tục phát triển các chức năng tự động hóa, nâng cao độ tin cậy và mở rộng phạm vi ứng dụng của hệ thống.

Em xin chân thành cảm ơn thầy Nguyễn Quốc Uy đã hướng dẫn và góp ý trong quá trình thực hiện đề tài.